

目录

- 1 实验目的**
- 2 实验平台及环境**
- 3 实验内容**
- 4 实验步骤与结果分析**
 - 4.1 前置步骤
 - 4.2 创建存储过程
 - 4.2.1 批量更新商品价格
 - 4.2.2 处理用户注册
 - 4.2.3 更新购物车
 - 4.2.4 清理超时未支付订单
 - 4.2.5 更新默认地址
 - 4.3 创建触发器
 - 4.3.1 检查商品价格
 - 4.3.2 检查商品库存
 - 4.3.3 商品分类删除保护
 - 4.4 管理存储过程
 - 4.5 管理触发器
- 5 实验总结与心得体会**

实验六 创建和管理存储过程/触发器

1 实验目的

1. 通过实验让学生熟悉并了解 Gauss DB数据库的基本机制与操作；
2. 通过创建和管理存储过程操作，让学生熟悉并了解 DAS 环境下如何使用 GaussDB；

2 实验平台及环境

本实验环境为华为云 GaussDB 数据库，具体的配置如表 2-1 所示：

表 2-1 实验环境

配置项	参数
华为云	软件版本：GaussDB v2.0-8.210.0
	设备型号：GaussDB 8 核 64 G
数据库	PostgreSQL

3 实验内容

本实验通过对存储过程/触发器管理等操作，让学生熟悉并了解 DAS 环境下如何使用 GaussDB 创建和调用及管理存储过程或触发器。

4 实验步骤与结果分析

为保证报告叙述的连续性，我将“实验步骤”与“实验结果分析”合并为一节，在每个操作步骤之后给出对应的结果和对实验结果的分析。

4.1 前置步骤

1. 登录华为云的 IAM 账户；
2. 登录 GaussDB 数据库实例；
 - (a) 在控制台选择并进入“云数据库 GaussDB”；
 - (b) 选择“gauss-6025-12”实例并登录；

4.2 创建存储过程

4.2.1 批量更新商品价格

要求：批量更新某个分类下的商品价格，比如将价格提高 50% 或打 8 折。

(1) 创建通用函数用于检索多级分类

由于在商品基本信息表中对应的分类信息是二级分类，所以必须特殊处理其一级分类。所以对于一级分类（父分类）而言，当分类的价格发生改变时，它的子分类的价格也会同步更新。

因此，对于商品价格的更新需要处理“多级分类”，一种思路是使用**递归查找**（华为云的 GaussDB 数据库支持），考虑兼容性和模块化，我先定义一个函数 `get_all_child_categories()` 来查找所有相关的分类 ID，对应的 SQL 语句见下文，执行结果如图 4-1 所示。

```
1 CREATE OR REPLACE FUNCTION get_all_child_categories(p_root_category_id INT)
2 RETURNS SETOF INT AS $$
3 BEGIN
4     -- 使用递归 CTE 查找自身及所有后代分类 ID
5     RETURN QUERY
6     WITH RECURSIVE CategoryHierarchy AS (
7         -- 基础查询
8         SELECT category_id
9         FROM "商品分类基本信息表"
10        WHERE category_id = p_root_category_id
11
12        UNION ALL
13
14        -- 递归查询
```

```

15      SELECT c.category_id
16      FROM "商品分类基本信息表" c
17      INNER JOIN CategoryHierarchy ch ON c.parent_id = ch.category_id
18  )
19      SELECT category_id FROM CategoryHierarchy;
20  END;
21  $$ LANGUAGE plpgsql;

```

注：创建函数的语法和创建存储过程是类似的，但实际测试发现，GaussDB 对于创建存储过程支持省略“\$\$...\$\$”来框定范围，而创建函数则必须使用其来包裹函数体。

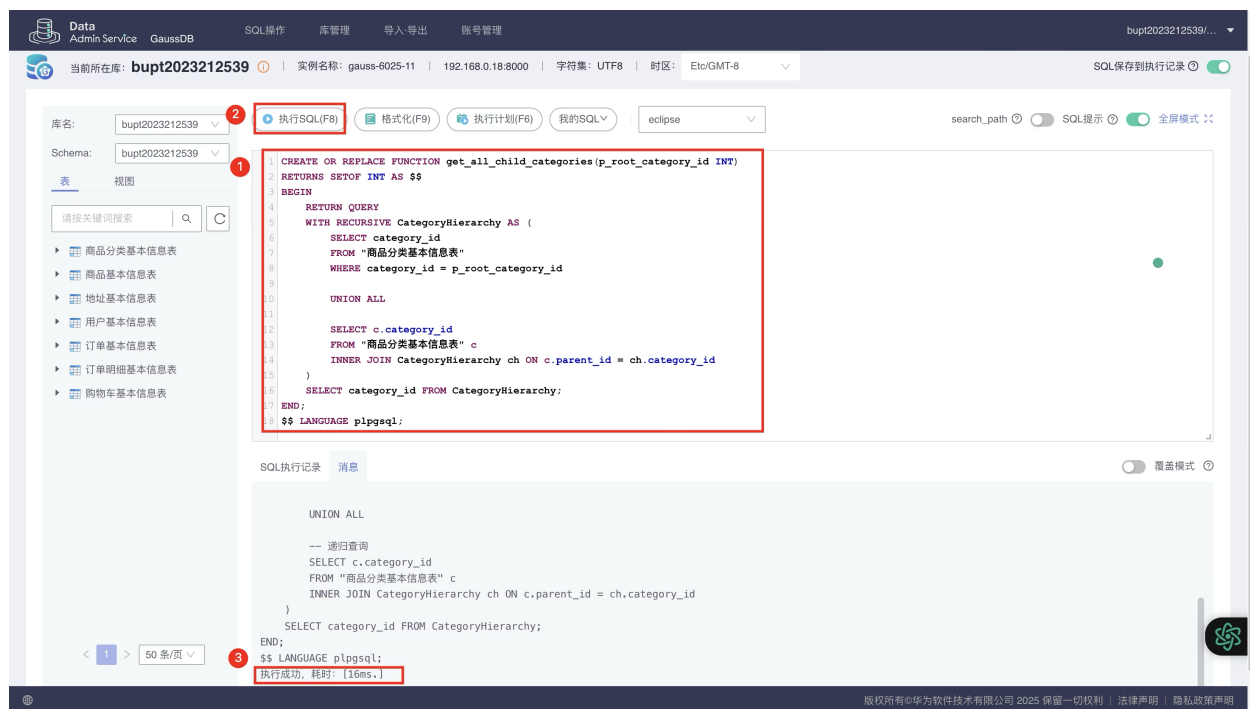


图 4-1 创建查找所有子分类函数

还有一种方法是使用自连接（和我在实验五中的思路一致，此处不再演示），因为实验提供的数据集中只有两级分类。

(2) 设计并创建存储过程

接下来，设计存储过程，命名为 `update_category_price`，该存储过程的目标是根据输入的分类 ID（参数命名为 `p_category_id`）和价格变化比例（`p_ratio`），更新该分类及其所有子分类下的商品价格，对应的 SQL 语句见下文，执行结果如图 4-2 所示。

```

1 CREATE OR REPLACE PROCEDURE update_category_price(
2     p_category_id INT,      -- 要更新的分类ID
3     p_ratio NUMERIC        -- 价格变化比例

```

```

4 )
5 AS
6 DECLARE
7     v_category_ids INT[];
8     v_rows_updated INT;
9 BEGIN
10     -- 1. 查找所有需要更新的分类 ID
11     SELECT ARRAY(SELECT get_all_child_categories(p_category_id)) INTO
v_category_ids;
12
13     -- 2. 更新商品价格
14     UPDATE "商品基本信息表"
15     SET
16         price = price * p_ratio,
17         update_time = NOW()
18     WHERE
19         category_id = ANY(v_category_ids);
20
21     -- 3. 输出操作结果用于调试
22     GET DIAGNOSTICS v_rows_updated = ROW_COUNT;
23     RAISE NOTICE '已更新分类 ID % 及其所有子分类下共 % 件商品的价格。',
24         p_category_id, v_rows_updated;
25 END;

```

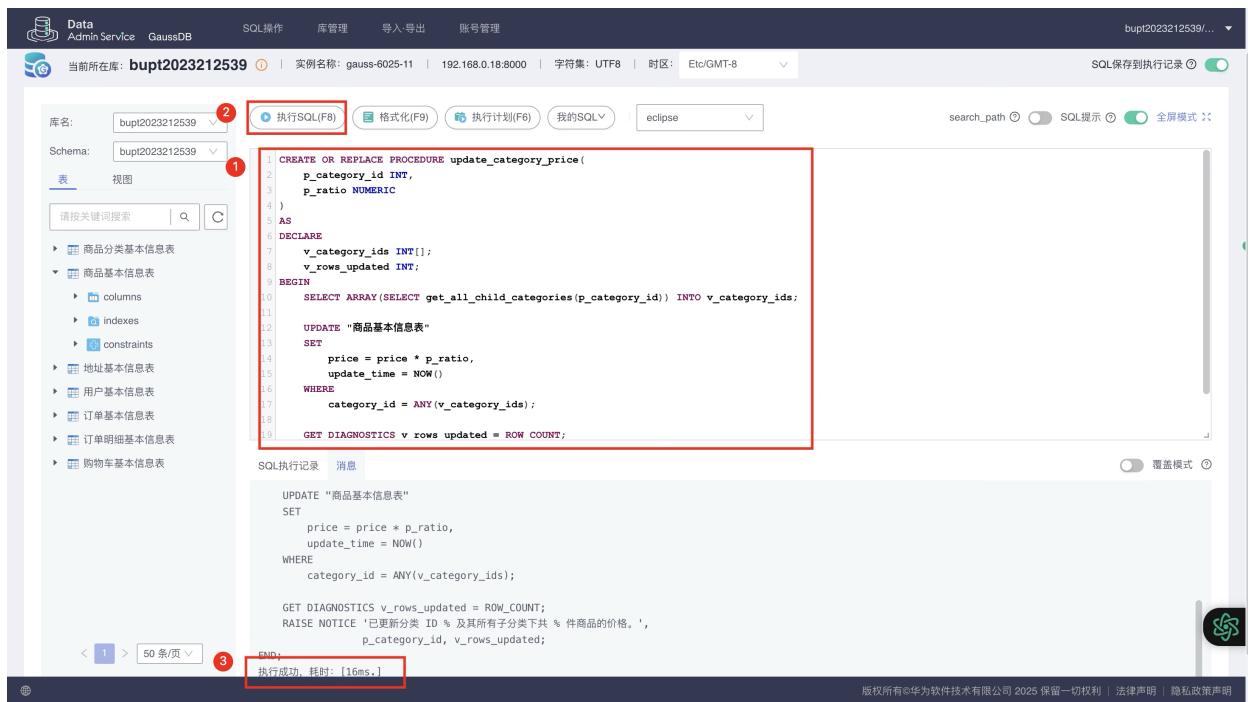


图 4-2 更新商品价格的存储过程

(3) 调用该存储过程并检查

(3.1) 批量更新分类编号为 10 的商品价格（二级分类，面食），将价格提高 50%，对应的 SQL 语句见下文，执行结果如图 4-3 和图 4-4 所示。

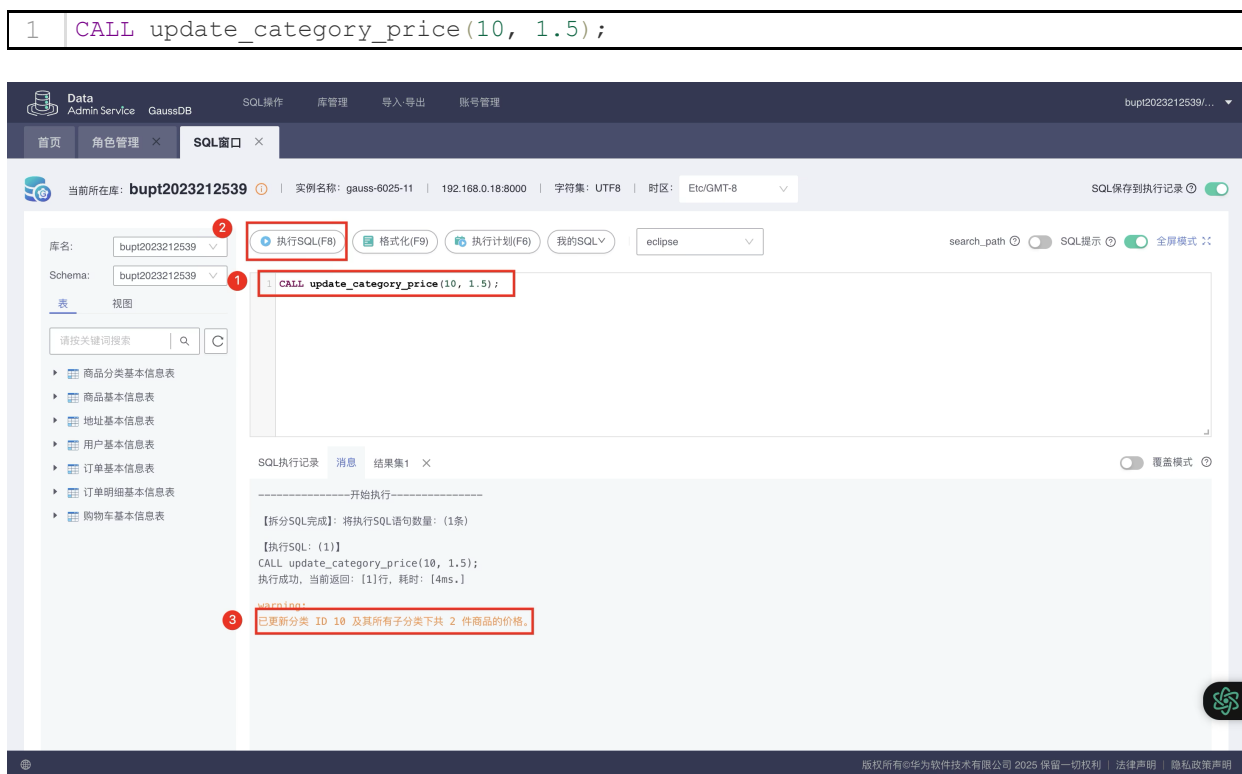


图 4-3 更新面食价格执行过程

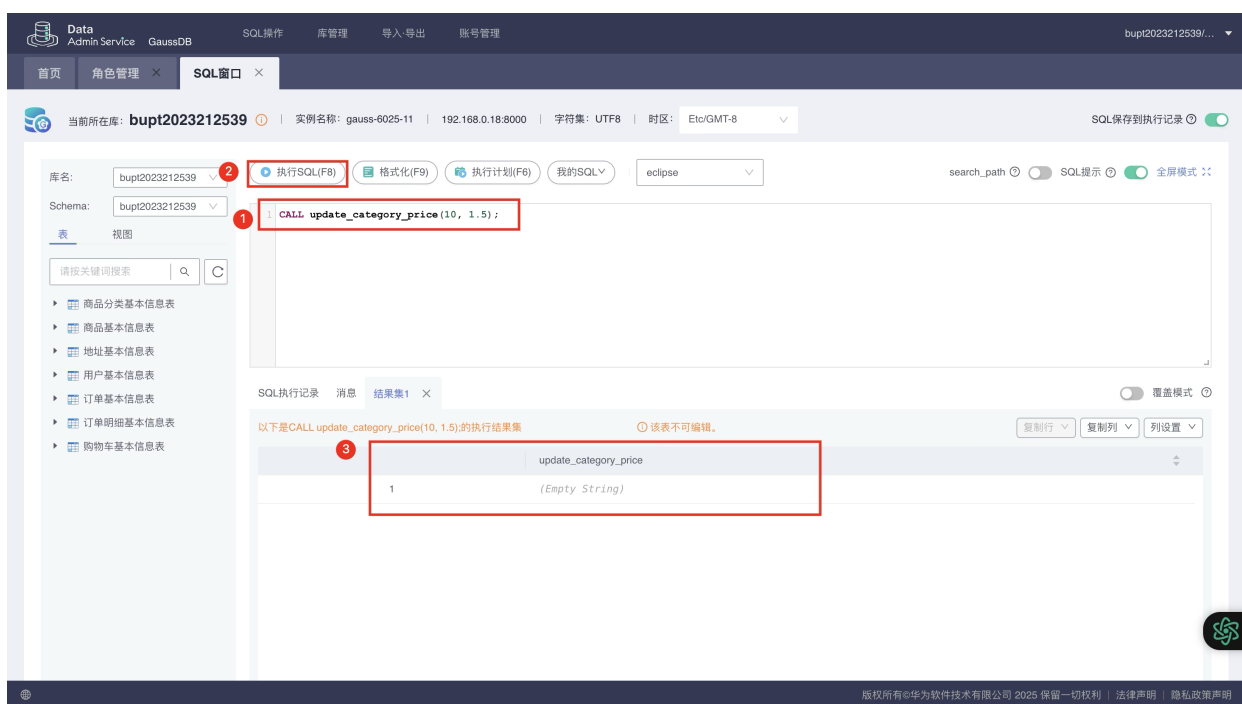


图 4-4 更新面食价格执行结果集

注: 存储过程 (`CREATE PROCEDURE`) 默认不会返回传统的查询结果集。它执行的是一系列事务操作而不是查询。对于 `CALL` 语句, 数据库通常会返回一个单列结果, 列名即为存储过程的名称, 内容为空, 表示命令执行成功。

因此，图 4-4 中，输出的结果集没有内容，列名为 `update_category_price`，即存储过程的名称，说明调用成功，从我添加的调试信息同样也可以验证，2 条面食商品的价格已经被更新成功。也可以打开数据表进行验证，如图 4-5 所示，两条商品的原价分别为“81”和“60”，现在已经正确提高 50%。

1	红烧牛肉面	精选牛肉，搭配手工拉面，汤底浓郁	121.50
2	老北京炸酱面	传统炸酱配方，搭配黄瓜丝和豆芽	90.00

图 4-5 更新后的数据表

(3.2) 批量更新分类编号为 1 的商品价格（一级分类，食品），将价格打八折，对应的 SQL 语句见下文，执行结果如图 4-6 所示。

1

CALL update_category_price(1, 0.8);

图 4-6 更新食品价格执行结果

共有 6 条食品类（包含子分类）商品被更新，由于篇幅限制，此处不再展示更新后的数据表结果。

4.2.2 处理用户注册

要求：查询当用户进行注册时，需要检查用户名是否存在，若不存在则插入新用户信息。

(1) 前置工作

由于“用户基本信息表”的主键 `user_id` 序列的当前值被重置或停留在了 1，当 `INSERT` 语句执行时，它向序列请求下一个 ID，序列返回 1，从而导致主键冲突 `Duplicate key value violates unique constraint`。

因此，在执行存储过程前，先通过手动将 `user_id` 序列的值调整为表中当前最大的 `user_id`，确保下一次插入从正确的 ID 开始，对应的 SQL 语句见下文。

```
1 SELECT SETVAL('用户基本信息表_user_id_seq', (SELECT MAX(user_id) FROM "用户基本信息表"));
```

(2) 设计并创建存储过程

这里，我设计存储过程并命名为 `register_user`，该存储过程用于查询用户名是否存在，若不存在则插入新用户，对应的 SQL 语句见下文，执行结果如图 4-7 所示。

```
1 CREATE OR REPLACE PROCEDURE register_user(  
2     p_username VARCHAR(50),  
3     p_password VARCHAR(255),  
4     p_email VARCHAR(100),  
5     p_phone VARCHAR(20) DEFAULT NULL,  
6     p_age INT DEFAULT NULL,  
7     p_gender VARCHAR(10) DEFAULT NULL  
8 )  
9 AS  
10 DECLARE  
11     v_user_count INT;  
12 BEGIN  
13     -- 1. 检查用户名是否存在  
14     SELECT COUNT(*) INTO v_user_count  
15     FROM "用户基本信息表"  
16     WHERE username = p_username;  
17  
18     IF v_user_count > 0 THEN  
19         RAISE EXCEPTION '用户名 "%" 已存在，注册失败。', p_username;  
20     ELSE  
21         -- 2. 插入新用户信息  
22         INSERT INTO "用户基本信息表" (username, password, email, phone, age,  
gender)  
23         VALUES (p_username, p_password, p_email, p_phone, p_age, p_gender);  
24  
25         RAISE NOTICE '用户 "%" 注册成功!', p_username;  
26     END IF;
```

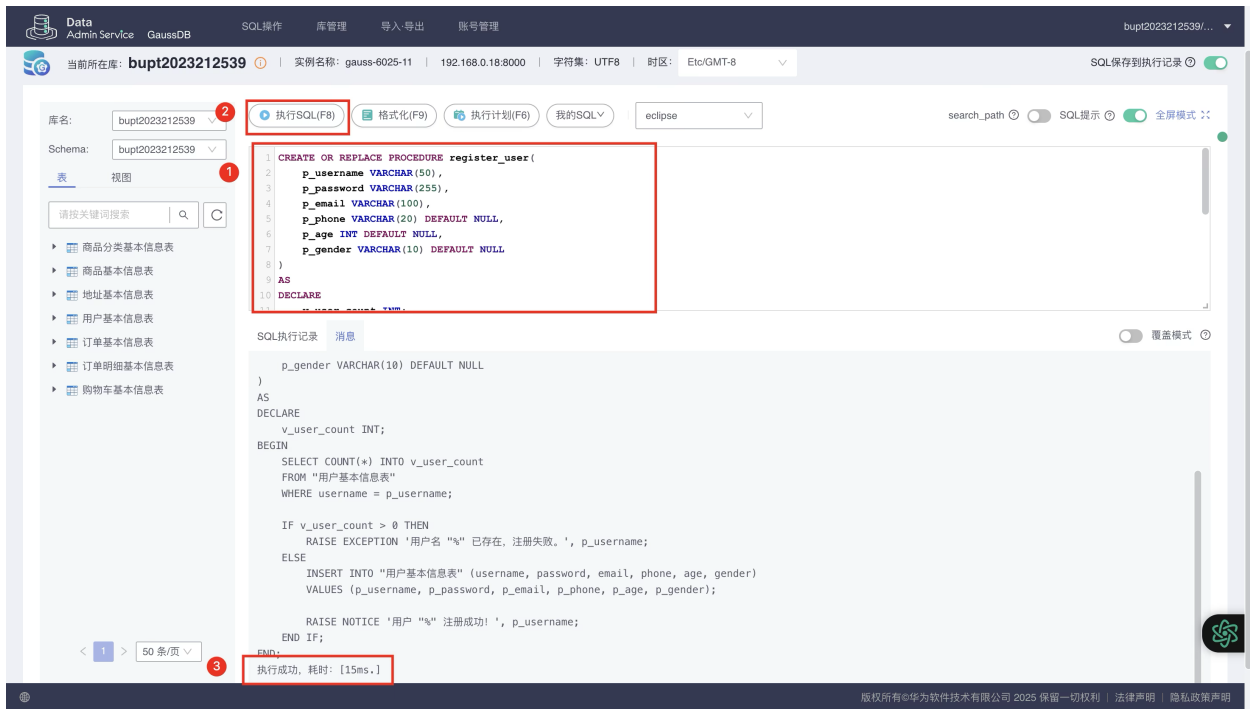


图 4-7 用户注册检查的存储过程

(3) 调用该存储过程并检查

(3.1) 若用户名已存在，例如，插入已经存在原表中 user_id 为 1 的用户数据，对应的 SQL 语句见下文，执行结果如图 4-8 所示，执行后正确抛出我在存储过程中定义的异常“**ERROR: 用户名 "张三" 已存在，注册失败。**”，验证了存储过程设计的正确性。

```
1 CALL register_user('张三', 'zhangsan123', 'zhangsan@example.com',  
  '13800138000', '28', '男');
```

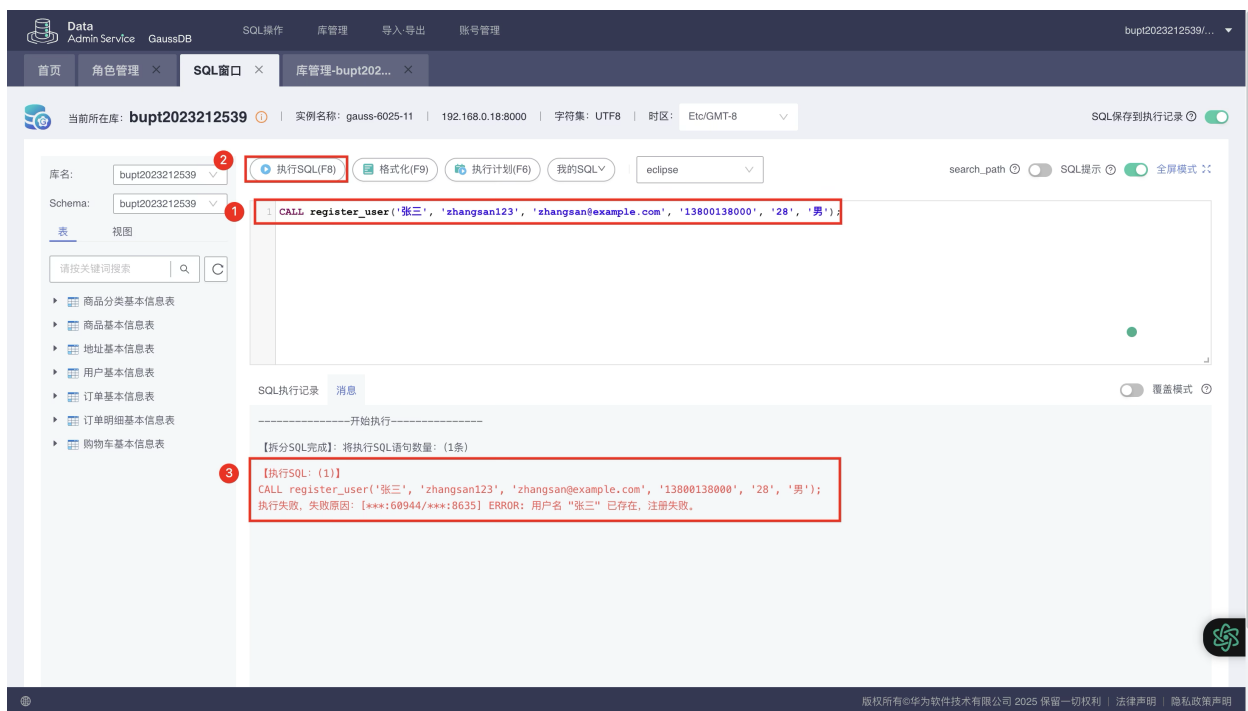


图 4-8 插入已存在用户的执行结果

(3.2) 若用户名不存在，则进行注册，例如，插入一条不存在于原表的用户数据，对应的 SQL 语句见下文，执行结果如图 4-9 所示，执行后正确抛出我在存储过程中定义的提出“用户 "bupt2023212539" 注册成功！”，验证了存储过程设计的正确性，通过查看数据表也可以验证，如图 4-10 所示，用户注册成功。

```
1 CALL register_user('bupt2023212539', 'bupt2023212539',  
  'bupt2023212539@email.com', '12345678900', '20', '男');
```

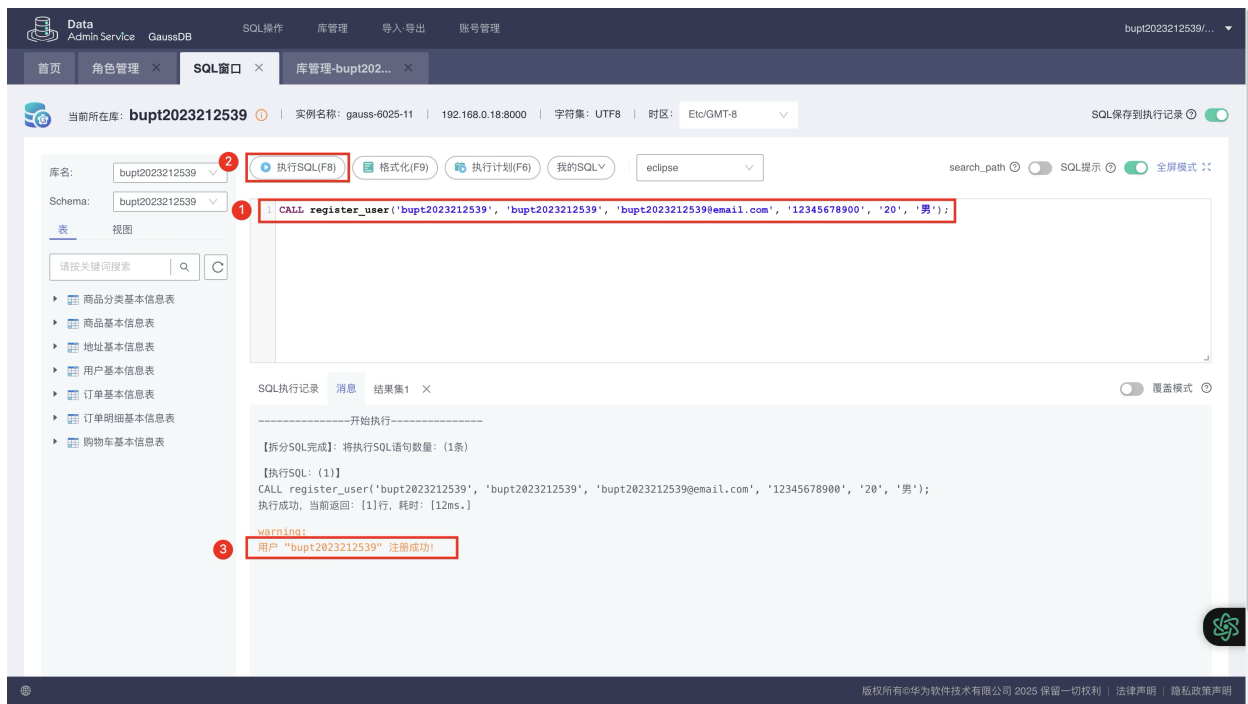


图 4-9 注册新用户的执行结果

107	109	bupt2023212539	bupt2023212539	bupt2023212539@email.co m
-----	-----	----------------	----------------	------------------------------

图 4-10 注册新用户后的数据表

4.2.3 更新购物车

要求：将商品添加到用户的购物车中。如果商品已存在，则更新数量；否则插入新记录。

(1) 前置准备

同上，将序列值设置为当前最大值，对应的 SQL 语句见下文。

```
1 SELECT setval ('购物车基本信息表_cart_id_seq', (SELECT MAX(cart_id) FROM "购物车  
基本信息表"));
```

另外，在实验二中，创建表的过程并没有保证 `user_id` 和 `product_id` 的组合唯一，不过从数据集和实际业务逻辑来看，应该是唯一的，即一条购物车记录对应用户购买的商品和对应数量，用户和商品的组合应该是唯一的，避免出现业务逻辑错误，更新表的约束，对应的 SQL 语句见下文。

```
1 -- 原表中存在重复数据，各个属性完全一致，所有直接删除一条  
2 DELETE FROM 购物车基本信息表 WHERE cart_id = 51;  
3 DELETE FROM 购物车基本信息表 WHERE cart_id = 40;  
4  
5 ALTER TABLE "购物车基本信息表"  
6 ADD CONSTRAINT uk_cart_user_product UNIQUE (user_id, product_id);
```

(2) 设计并创建存储过程

这里，我设计存储过程并命名为 `add_to_cart`，该存储过程用于将商品添加到用户的购物车中。如果商品已存在，则更新数量；否则插入新记录。其中，用户输入的参数包括 `user_id`，`product_id` 和 `quantity`，其中 `quantity` 表示新增的已存在商品数量，或新增商品的数量。对应的 SQL 语句见下文，执行结果如图 4-11 所示。

```
1 CREATE OR REPLACE PROCEDURE add_to_cart(  
2     p_user_id INT,  
3     p_product_id INT,  
4     p_quantity INT  
5 )  
6 AS
```

```

7 BEGIN
8     -- 1. 如果购物车中已存在该商品，则增加数量
9     UPDATE "购物车基本信息表"
10    SET quantity = quantity + p_quantity,
11        create_time = NOW()
12    WHERE user_id = p_user_id AND product_id = p_product_id;
13
14    -- 2. 检查是否进行了更新
15    IF NOT FOUND THEN
16        -- 如果没有更新记录（即商品不存在），则插入新记录
17        INSERT INTO "购物车基本信息表" (user_id, product_id, quantity)
18        VALUES (p_user_id, p_product_id, p_quantity);
19
20        RAISE NOTICE '用户 % 的购物车插入商品 %, 数量 %。', p_user_id,
21            p_product_id, p_quantity;
22    ELSE
23        -- 已经更新了商品数量
24        RAISE NOTICE '用户 % 的购物车更新商品 %, 新增数量 %。', p_user_id,
25            p_product_id, p_quantity;
26    END IF;
27 END;

```

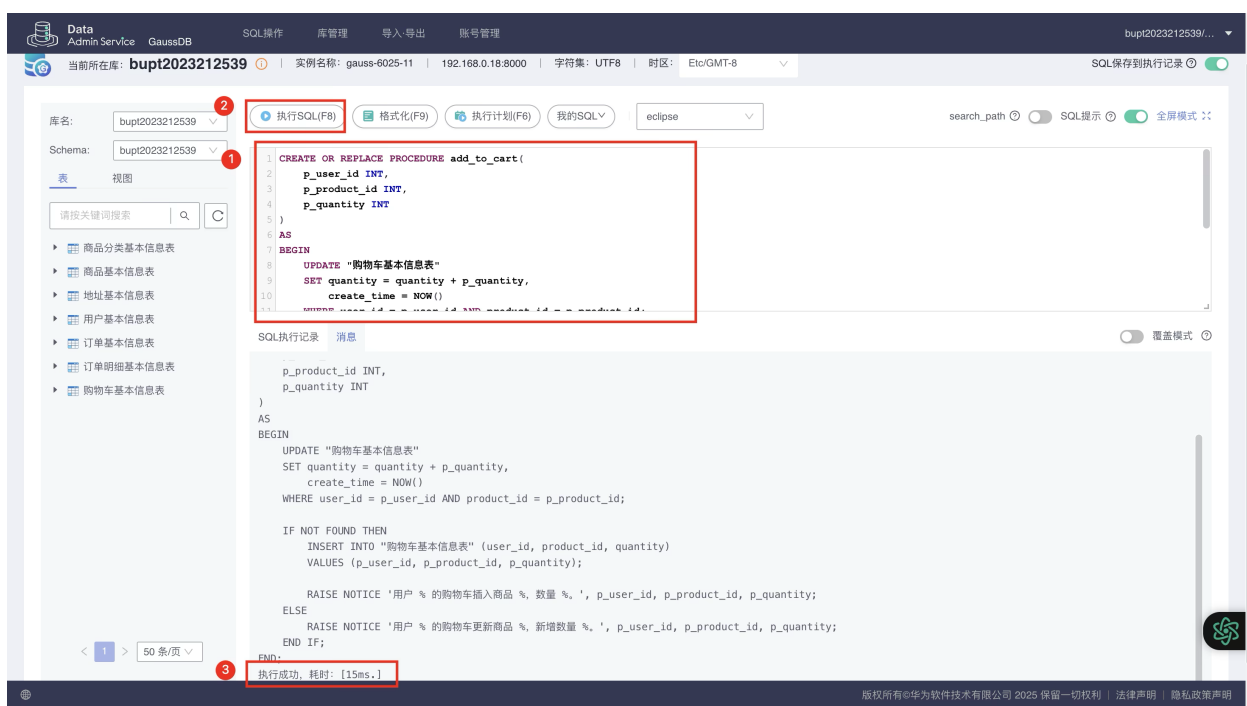


图 4-11 用户购物车更新的存储过程

(3) 调用该存储过程并检查

(3.1) 若添加已经在购物车的商品，则更新商品数量，例如，`user_id = 87` 的用户的购物车中已经有了两件 `product_id = 7` 的商品，现在又需要加购 3 件。对应的 SQL 语句见下文，执行结果如图 4-12 所示。

执行后正确抛出我在存储过程中定义的提示“用户 87 的购物车更新商品 7，新增数量 3。”，验证了存储过程设计的正确性，同时，通过查看更新后的数据表，如图 4-13 所示，也可以验证，用户 87 原来购买了 2 件商品 7，现在数量更新为 5 件。

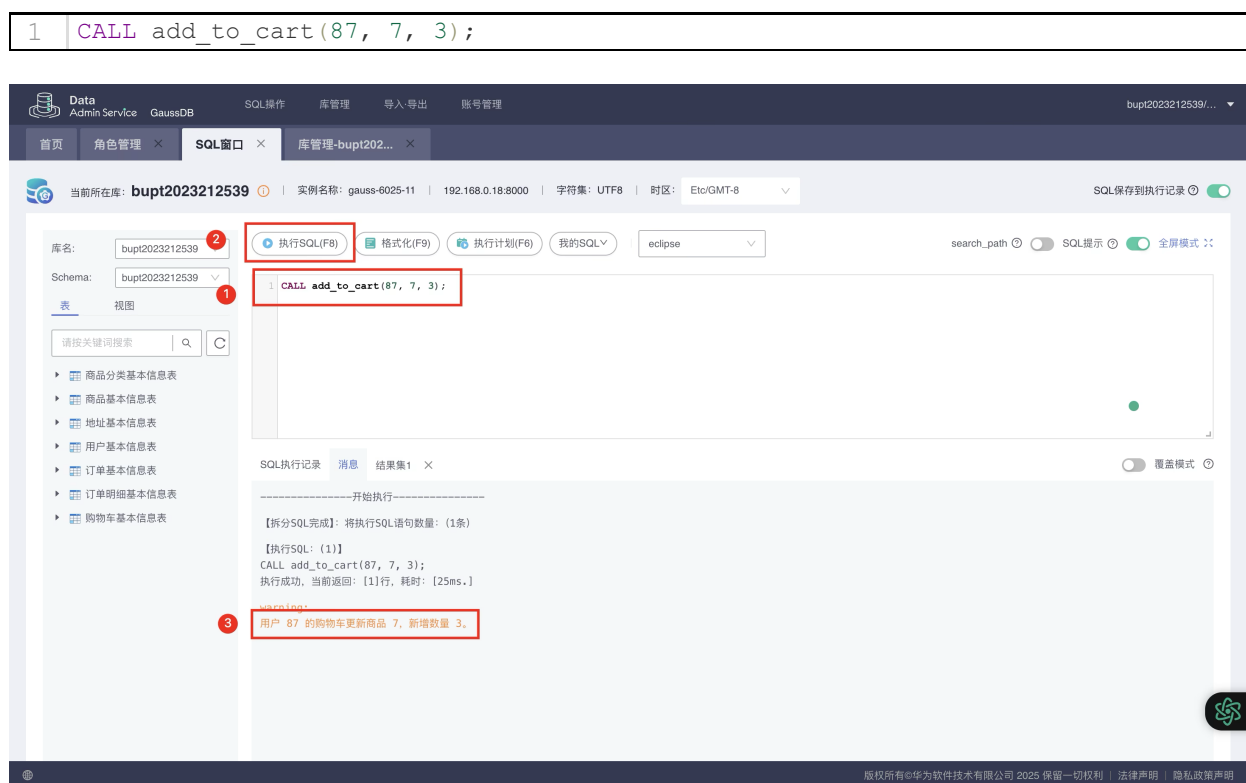


图 4-12 新增已存在商品数量的执行结果

cart_id	user_id	product_id	quantity
2	87	7	5

图 4-13 新增已存在商品数量后的数据表

(3.2) 若添加不在购物车的商品，则插入数据，例如，`user_id = 87` 的用户现在需要购买 2 件 `product_id = 18` 的商品，对应的 SQL 语句见下文，执行结果如图 4-14 所示，执行后正确抛出我在存储过程中定义的提示“用户 87 的购物车插入商品 18，数量 2。”，验证了存储过程设计的正确性，

```
1 CALL add_to_cart(87, 18, 2);
```

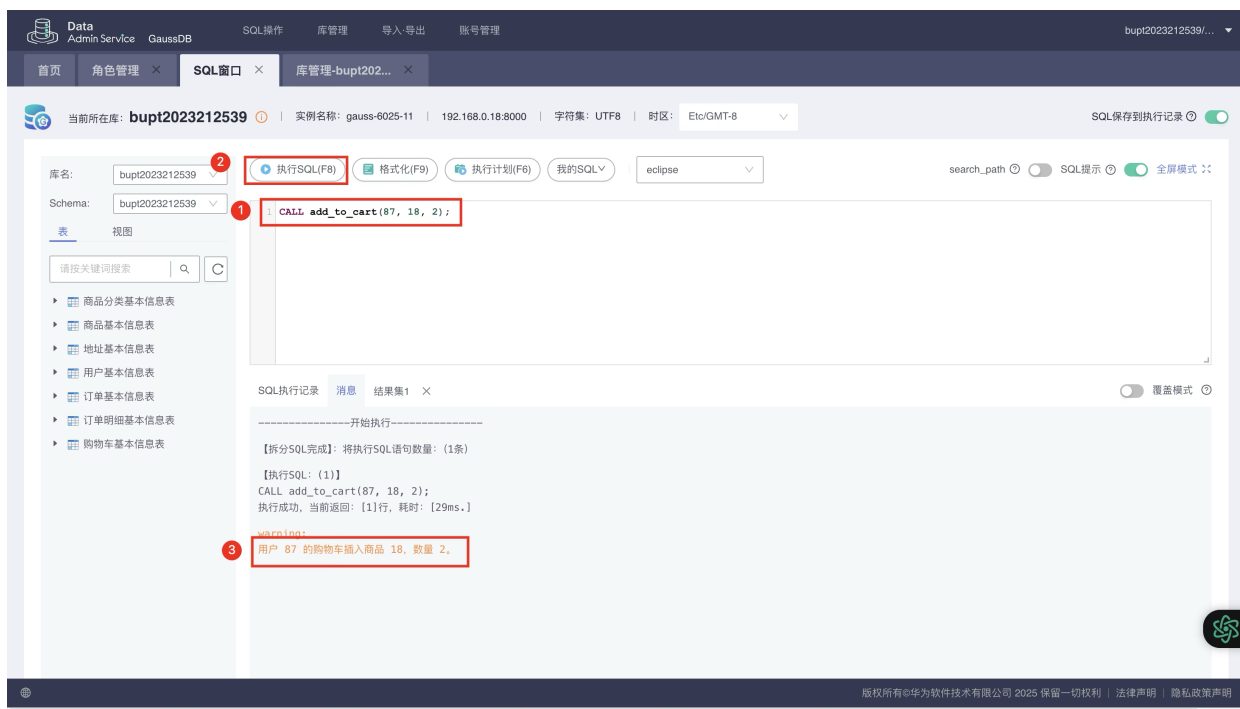


图 4-14 新增之前不存在购物车的商品的执行结果

4.2.4 清理超时未支付订单

要求：删除超过指定天数（用户指定的输入参数）的未支付订单。（假设未支付订单的 `total_amount = 0`）

（1）设计并创建存储过程

我设计的存储过程命名为 `delete_unpaid_orders`，用于删除超过指定天数的未支付订单（假设未支付订单的 `total_amount` 为 0），对应的 SQL 语句见下文，执行结果如图 4-15 所示。

```

1 CREATE OR REPLACE PROCEDURE delete_unpaid_orders(
2     p_days_threshold INT -- 指定的天数
3 )
4 AS
5 DECLARE
6     v_rows_deleted INT;
7 BEGIN
8     -- 删除满足条件的订单：
9     DELETE FROM "订单基本信息表"
10    WHERE total_amount = 0
11        AND create_time < NOW() - INTERVAL '1 day' * p_days_threshold;
12
13    GET DIAGNOSTICS v_rows_deleted = ROW_COUNT;
14
15    RAISE NOTICE '已删除 % 个超过 % 天的未支付订单。', v_rows_deleted,
    p_days_threshold;

```

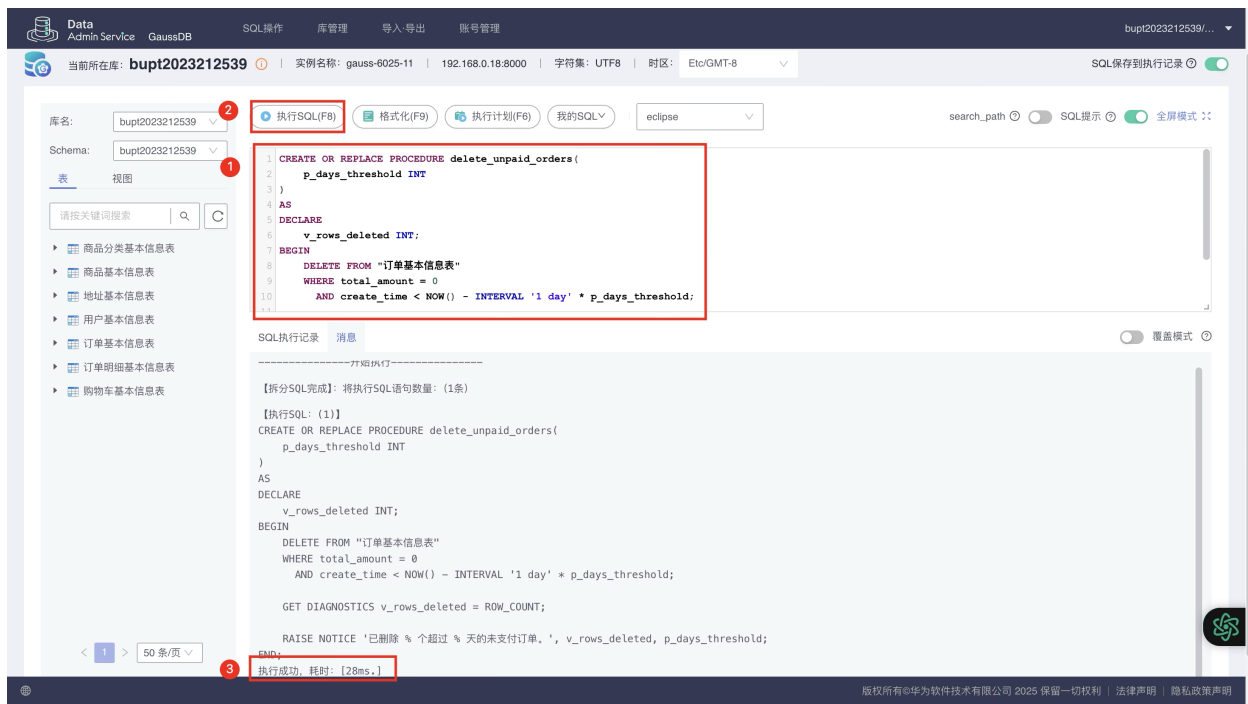


图 4-15 删除逾期未支付订单的存储过程

(2) 调用该存储过程并检查

删除超过 30 天的未支付订单，对应的 SQL 语句见下文，执行结果如图 4-16 所示，由于“订单基本信息表”中的不存在未支付订单，所以输出提示为“已删除 0 个超过 30 天的未支付订单。”。

```
1 CALL delete_unpaid_orders(30);
```

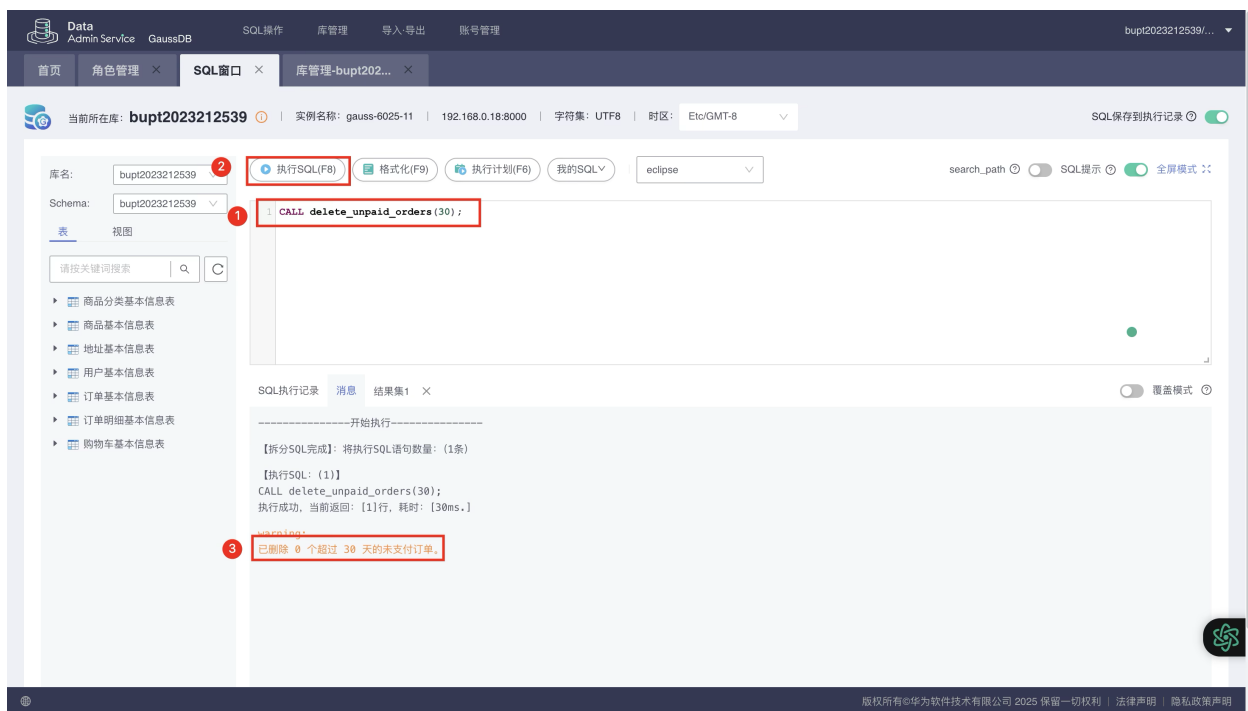


图 4-16 删除逾期未支付订单的执行结果

4.2.5 更新默认地址

要求：更新用户的默认地址，并确保只有一个默认地址。

(1) 设计并创建存储过程

我设计的存储过程设计命名为 `update_default_address`，用于更新用户的默认地址，并确保该用户只有一个默认地址。对应的 SQL 语句见下文，执行结果如图 4-17 所示。

```

1 CREATE OR REPLACE PROCEDURE update_default_address(
2     p_user_id INT,
3     p_address_id INT
4 )
5 AS
6 BEGIN
7     -- 1. 将该用户所有地址的 is_default 字段设置为 FALSE
8     UPDATE "地址基本信息表"
9     SET is_default = FALSE
10    WHERE user_id = p_user_id;
11
12    -- 2. 将指定地址的 is_default 字段设置为 TRUE
13    UPDATE "地址基本信息表"
14    SET is_default = TRUE
15    WHERE user_id = p_user_id AND address_id = p_address_id;
16
17    -- 检查是否更新成功
18    IF NOT FOUND THEN
  
```

```

19         RAISE EXCEPTION '用户 % 下找不到地址 ID %, 无法设置默认地址。', p_user_id,
        p_address_id;
20     ELSE
21         RAISE NOTICE '用户 % 的默认地址已成功设置为地址 ID %。', p_user_id,
        p_address_id;
22     END IF;
23 END;

```

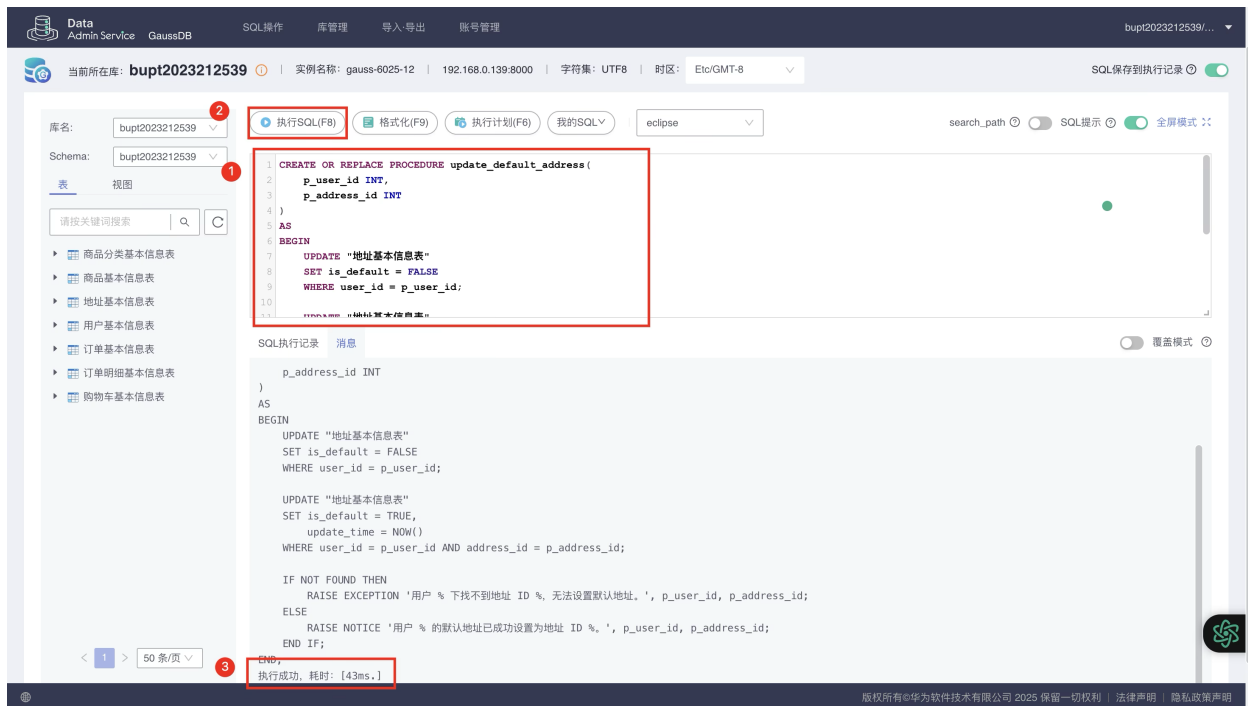


图 4-17 更新默认地址的存储过程

(2) 调用该存储过程并检查

设置用户编号为 1 的地址编号为 1 的地址为默认地址，对应的 SQL 语句见下文，执行结果如图 4-18 所示，输出提示“用户 1 的默认地址已成功设置为地址 ID 1。”，验证了存储过程创建的正确性。

```

1 CALL update_default_address(1, 1);

```

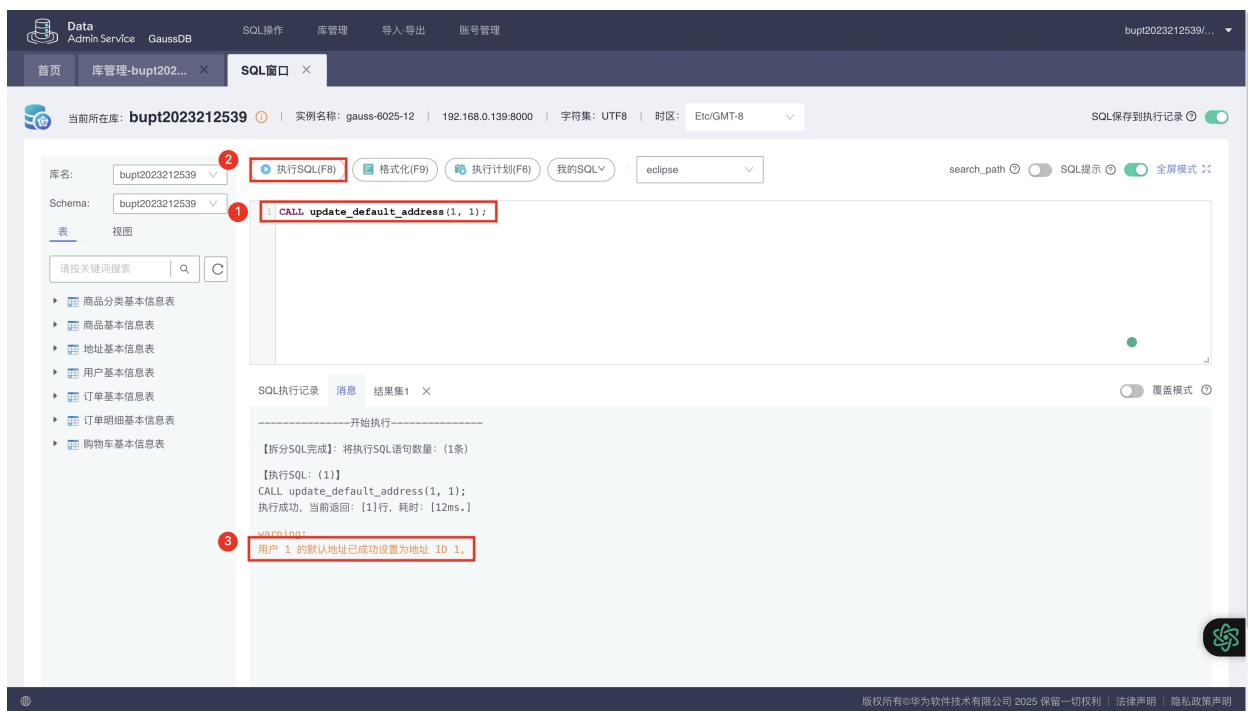


图 4-18 更新默认地址的执行结果

4.3 创建触发器

4.3.1 检查商品价格

要求：插入或更新商品时，确保价格大于0，否则不允许添加新商品。

(1) 创建触发器函数

首先，创建触发器函数，命名为 `check_product_price`，用于检查插入或更新商品时，价格是否大于 0，否则阻止操作并报错。对应的 SQL 语句见下文，执行结果如图 4-19 所示。

```

1 CREATE OR REPLACE FUNCTION check_product_price()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- 检查新记录的价格是否小于或等于 0
5     IF NEW.price <= 0 THEN
6         -- 抛出异常，阻止 INSERT 或 UPDATE 操作
7         RAISE EXCEPTION '尝试插入或更新的价格为 % 出错，商品价格必须大于 0。',
8         NEW.price;
9     END IF;
10    RETURN NEW;
11 END;
12 $$ LANGUAGE plpgsql;

```

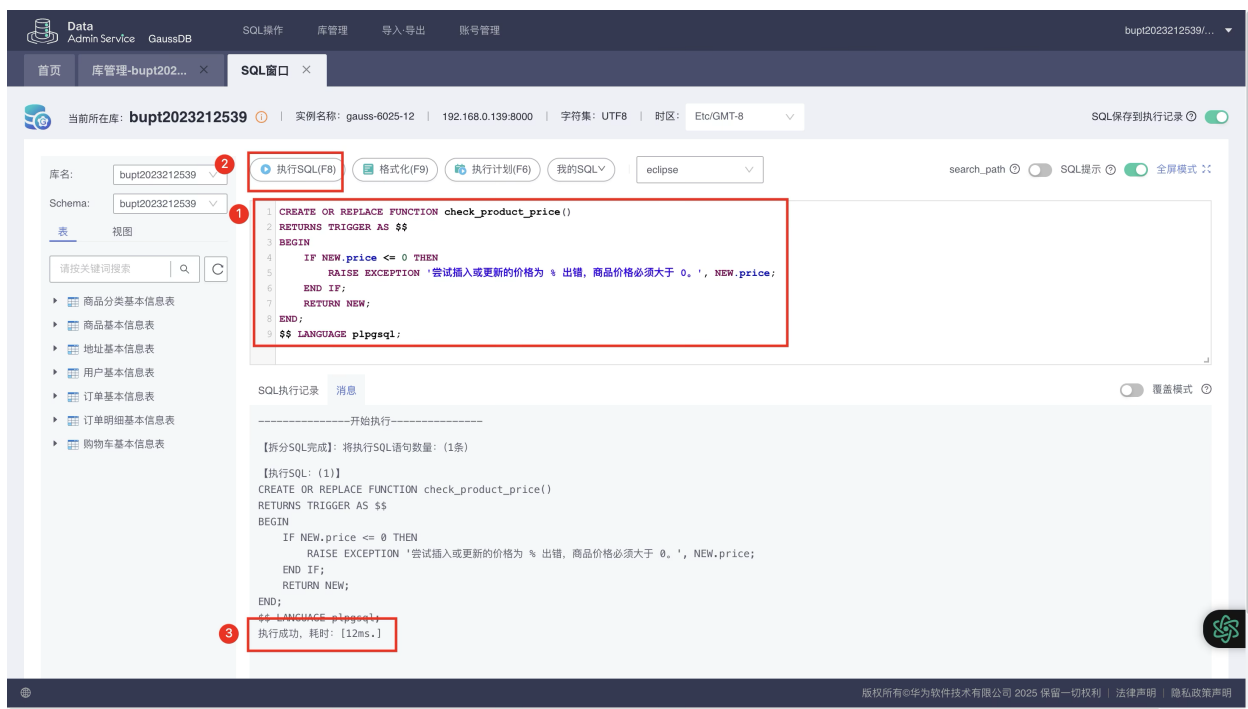


图 4-19 触发器函数

(2) 创建触发器并绑定函数

创建触发器 `product_price_trigger`，并绑定到触发器函数。对应的 SQL 语句见下文，执行结果如图 4-20 所示。

```
1 CREATE TRIGGER product_price_trigger
2 BEFORE INSERT OR UPDATE ON "商品基本信息表"
3 FOR EACH ROW
4 EXECUTE PROCEDURE check_product_price();
```

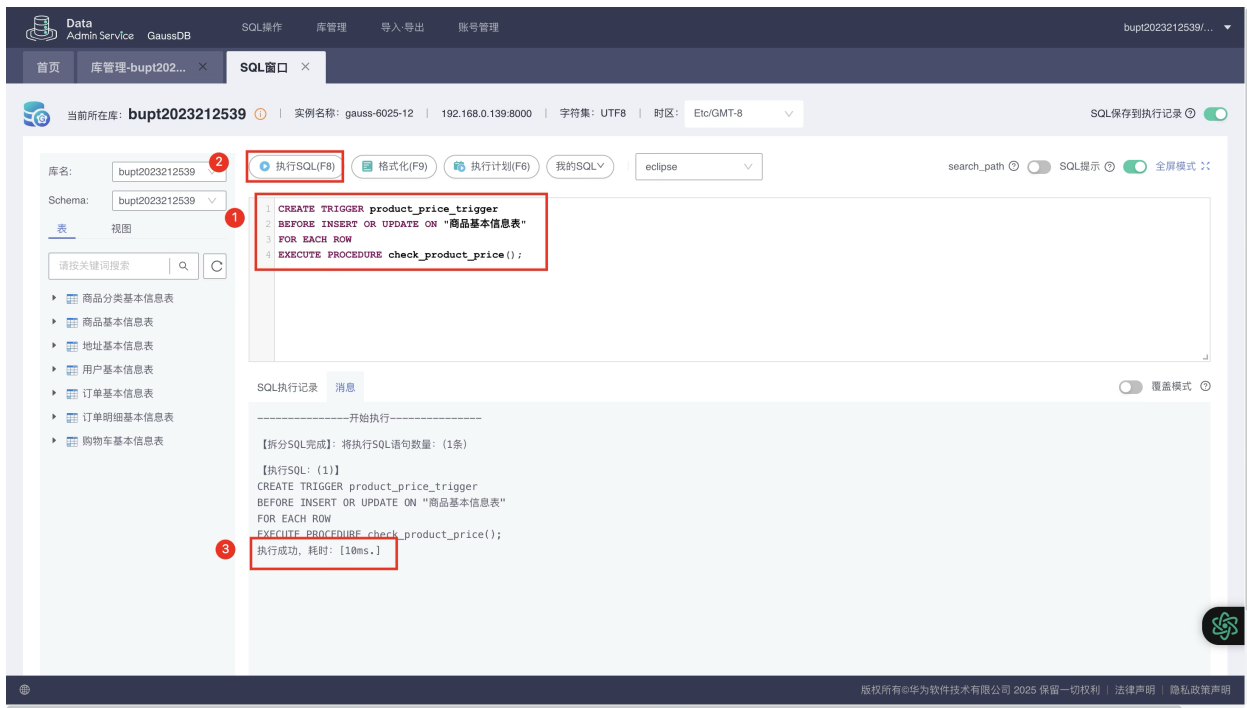


图 4-20 创建触发器

(3) 触发触发器

(3.1) 插入合法商品，对应的 SQL 语句见下文，执行结果如图 4-21 所示，价格为 10.99 的测试商品正确插入数据表。

```
1 INSERT INTO "商品基本信息表" (name, description, price, stock, category_id)
2 VALUES ('合法测试商品', '该商品价格大于0', 10.99, 100, 5);
```

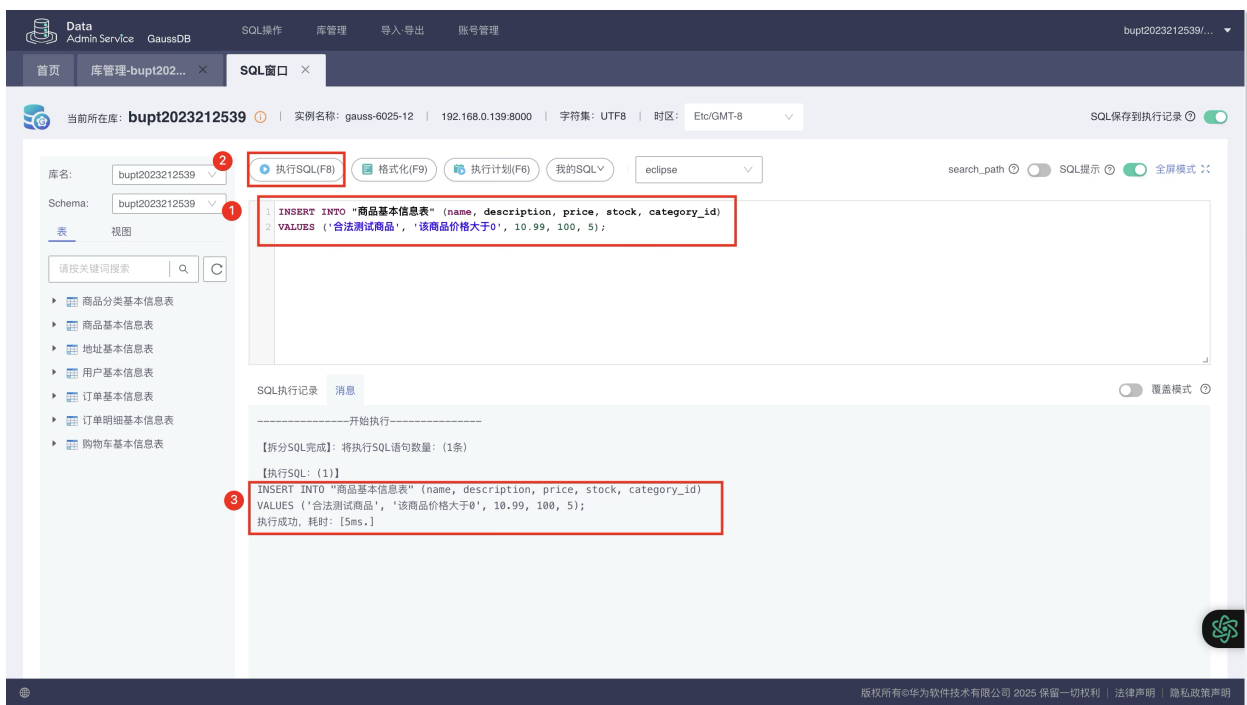


图 4-21 插入合法商品

(3.2) 插入不合法商品，对应的 SQL 语句见下文，执行结果如图 4-22 所示，插入价格为 0 的测试商品后，触发触发器，并调用我定义的函数抛出异常“**ERROR: 尝试插入或更新的价格为 0.00 出错，商品价格必须大于 0。**”，与预期结果一致，正确拦截了插入操作。

```
1 INSERT INTO "商品基本信息表" (name, description, price, stock, category_id)
2 VALUES ('不合法测试商品', '价格为0', 0.00, 10, 5);
```

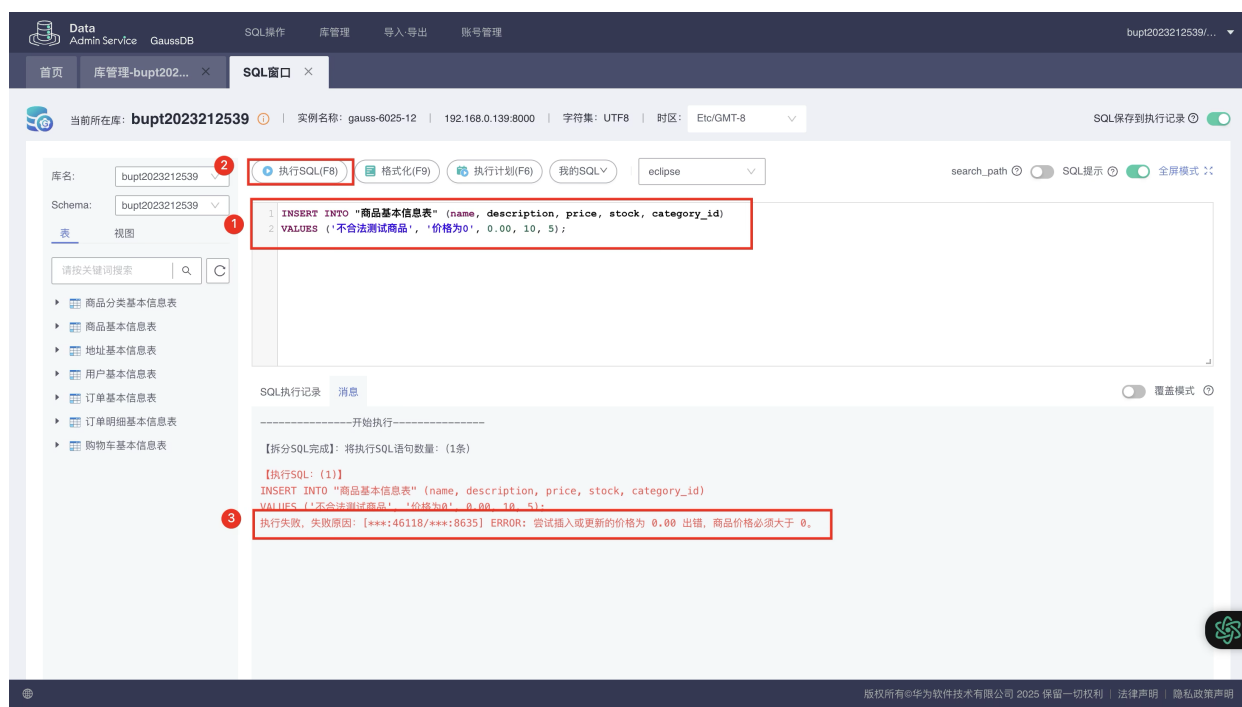


图 4-22 插入不合法商品

(3.3) 更新商品价格为负数，例如，这里更新 `product_id` 为 4 的商品的价格为 -5，对应的 SQL 语句见下文，执行结果如图 4-23 所示，更新商品价格为负数后，触发触发器，并调用我定义的函数抛出异常“**ERROR: 尝试插入或更新的价格为 -5.00 出错，商品价格必须大于 0。**”，与预期结果一致，正确拦截了更新操作。

```
1 UPDATE "商品基本信息表"
2 SET price = -5.00
3 WHERE product_id = 4;
```

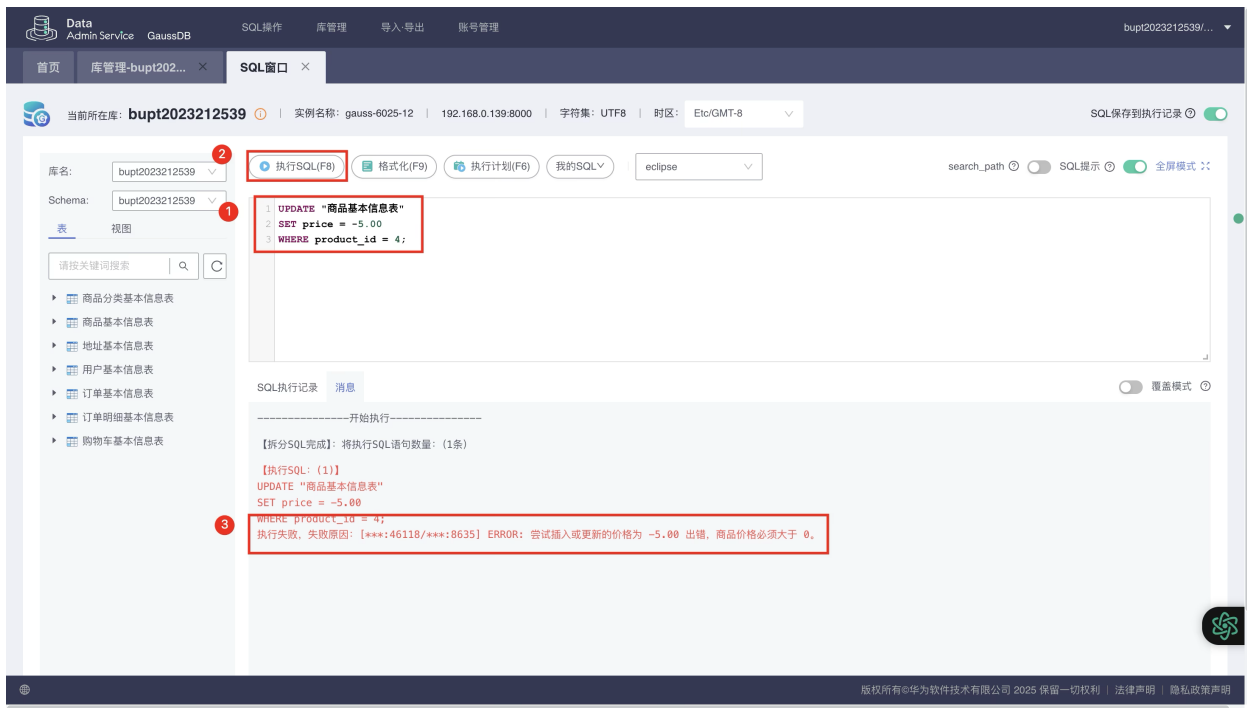


图 4-23 更新商品价格为负数

4.3.2 检查商品库存

要求：用户进行下单时，需对商品库存进行检查，若库存充足则插入新数据，若不充足则提供报错信息。下单时或添加购物车时需要检查库存是否充足，若不足则阻止操作。

(1) 创建触发器函数

这个要求涉及两个表的两个不同的触发器：一个用于购物车，一个用于订单明细。但是，对于库存的检查逻辑是一致的，都通过“商品基本信息表”中的信息判断，因此，触发器函数只需创建一个，名为 `check_stock_logic`；对应的 SQL 语句见下文，执行结果如图 4-24 所示。

```
1 CREATE OR REPLACE FUNCTION check_stock_logic()  
2 RETURNS TRIGGER AS $$  
3 DECLARE  
4     v_current_stock INT;  
5 BEGIN  
6     -- 1. 获取商品的当前库存  
7     SELECT stock INTO v_current_stock  
8     FROM "商品基本信息表"  
9     WHERE product_id = NEW.product_id;  
10  
11     -- 2. 检查库存是否充足  
12     IF NEW.quantity > v_current_stock THEN  
13         RAISE EXCEPTION '商品库存不足。商品ID: %, 需求数量: %, 当前库存: %',
```

```

14         NEW.product_id, NEW.quantity, v_current_stock;
15     END IF;
16
17     RETURN NEW;
18 END;
19 $$ LANGUAGE plpgsql;

```

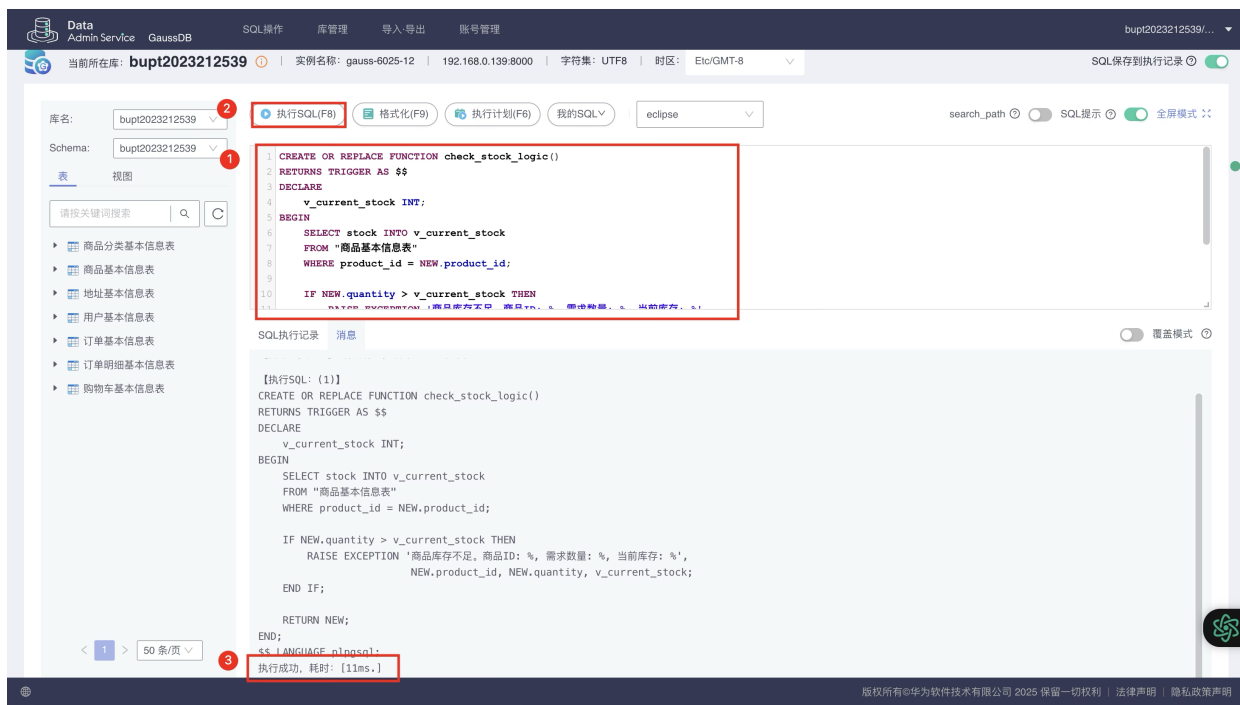


图 4-24 检查库存函数

(2) 创建触发器并绑定函数

对于购物车库存检查，创建触发器 `cart_stock_check_trigger`，并绑定到触发器函数；对于订单明细库存检查，创建触发器 `order_item_stock_check_trigger`，并绑定到触发器函数。对应的 SQL 语句见下文，执行结果如图 4-25 所示。

注：按照实际业务逻辑，这里应该再创建一个 AFTER 触发器，用于在下单成功后扣减库存，不过由于实验指导书未要求，省略实现。

```

1 CREATE TRIGGER cart_stock_check_trigger
2 BEFORE INSERT OR UPDATE ON "购物车基本信息表"
3 FOR EACH ROW
4 EXECUTE PROCEDURE check_stock_logic();
5
6 CREATE TRIGGER order_item_stock_check_trigger
7 BEFORE INSERT ON "订单明细基本信息表"
8 FOR EACH ROW
9 EXECUTE PROCEDURE check_stock_logic();

```

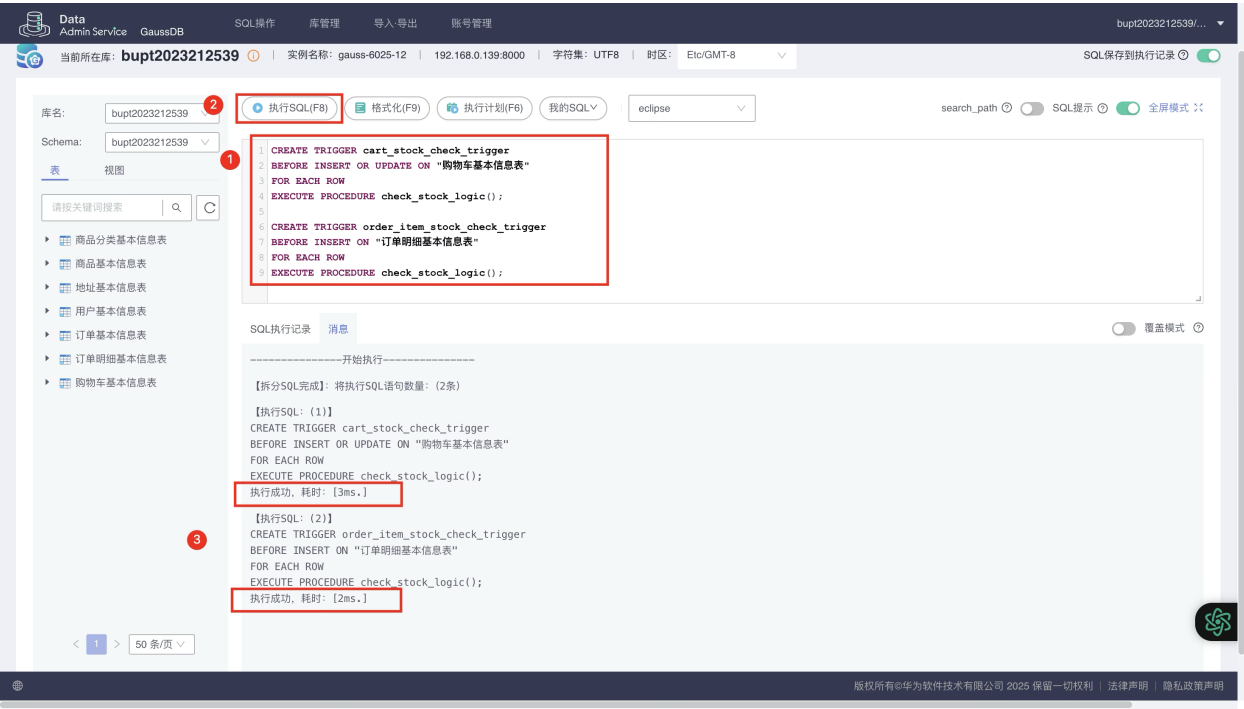


图 4-25 创建库存检查触发器

(3) 触发触发器

(3.1) 添加购物车数据（商品的当前库存充足），例如，商品 ID 为 4 的商品现在库存为 200，ID 为 1 的用户下单了 10 份，对应的 SQL 语句见下文，执行结果如图 4-26 所示。

(3.2) 添加购物车数据库存不足时，例如，商品 ID 为 4 的商品现在库存为 200，ID 为 1 的用户下单了 210 份，对应的 SQL 语句见下文，执行结果如图 4-26 所示。

```
1 CALL add_to_cart(1, 4, 10);
2
3 CALL add_to_cart(1, 4, 210);
```

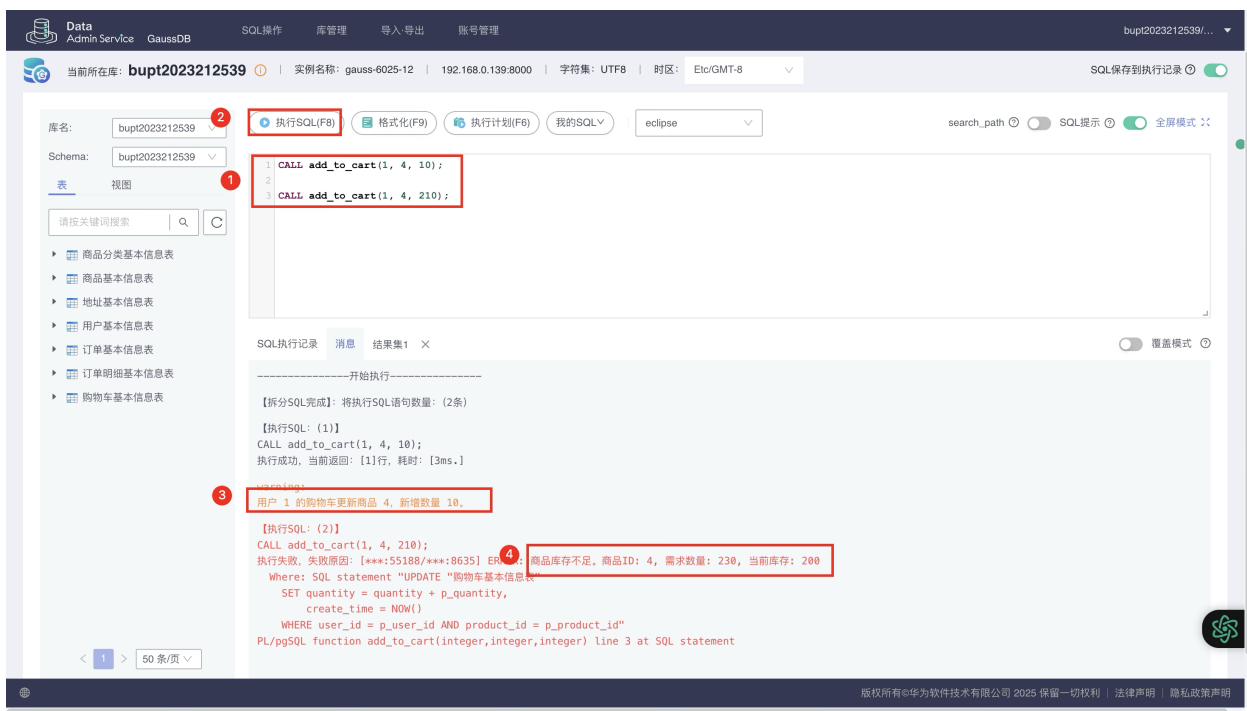


图 4-26 添加购物车库存充足/不充足

如图 4-26 所示，库存充足时，正确触发触发器并调用函数，抛出“用户 1 的购物车更新商品 4，新增数量 10。”，正确更新购物车。当库存不充足时，抛出异常“ERROR: 商品库存不足。商品ID: 4, 需求数量: 230, 当前库存: 200”。

这里，需求数量为 230 是因为我的 `add_to_cart` 是在原有购物车记录上进行更新，原来有 10 件商品 4，第一次更新，增加了 10 变为 20，第三次试图进行更新，增加 210 变为 230，但被触发器正确阻止。

(3.3) 添加订单明细数据（商品的当前库存充足），例如，ID 为 1 的订单又追加了 10 件 ID 为 4 的商品，对应的 SQL 语句见下文，执行结果如图 4-27 所示。

(3.4) 添加订单明细数据（商品的当前库存不足），例如，ID 为 1 的订单又追加了 210 件 ID 为 4 的商品，对应的 SQL 语句见下文，执行结果如图 4-27 所示。

```

1 SELECT setval ('"订单明细基本信息表_order_item_id_seq"', (SELECT
2 MAX(order_item_id) FROM "订单明细基本信息表"));
3 INSERT INTO "订单明细基本信息表" (order_id, product_id, quantity, price)
4 VALUES (1, 4, 10, 69.00);
5
6 INSERT INTO "订单明细基本信息表" (order_id, product_id, quantity, price)
7 VALUES (1, 4, 210, 69.00);

```

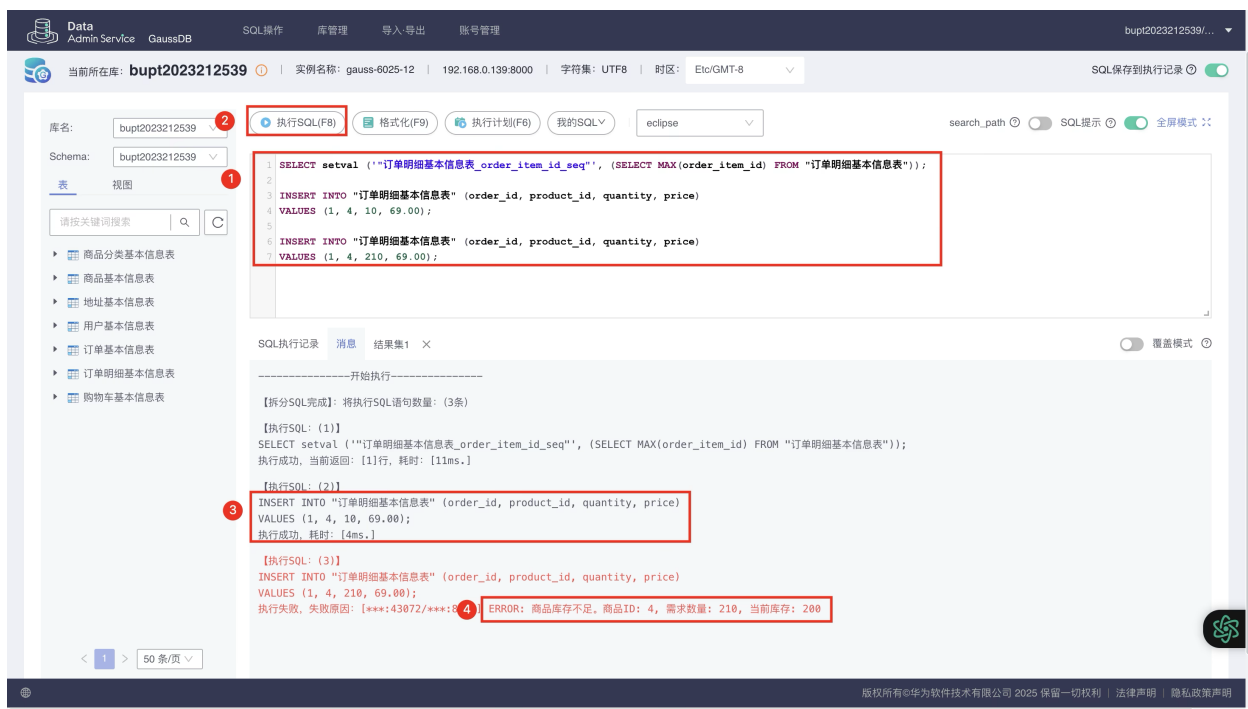


图 4-27 添加订单明细库存充足/不充足

如图 4-27 所示，库存充足时，正确插入新数据。当库存不充足时，抛出异常“**ERROR: 商品库存不足。商品ID: 4, 需求数量: 210, 当前库存: 200**”。

4.3.3 商品分类删除保护

要求：商品分类删除保护（若某个分类下有商品，则不允许删除该分类）

(1) 创建触发器函数

我在之前创建存储过程时，已经创建了 `get_all_child_categories` 函数获取父分类及其所有子分类的 ID 集合，因此，在删除分类时，需要检查当前分类及其所有子分类下是否有商品，可利用该函数获取所有受影响的分类 ID 集合，并设计触发器函数名为 `category_delete_protection`，对应的 SQL 语句见下文，执行结果如图 4-28 所示。

```

1 CREATE OR REPLACE FUNCTION category_delete_protection()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     v_product_count INT;
5 BEGIN
6     -- 1. 检查被删除的分类 (OLD.category_id) 及其所有子分类下是否有商品。
7     SELECT COUNT(p.product_id) INTO v_product_count
8     FROM "商品基本信息表" p
9     WHERE p.category_id IN (SELECT
get_all_child_categories(OLD.category_id));

```

```

10
11      -- 2. 如果找到商品，则阻止删除并抛出异常。
12      IF v_product_count > 0 THEN
13          RAISE EXCEPTION '无法删除分类 ID %。该分类或其子分类下仍有 % 件商品关联。',
14                          OLD.category_id, v_product_count;
15      END IF;
16
17      RETURN OLD;
18  END;
19  $$ LANGUAGE plpgsql;

```

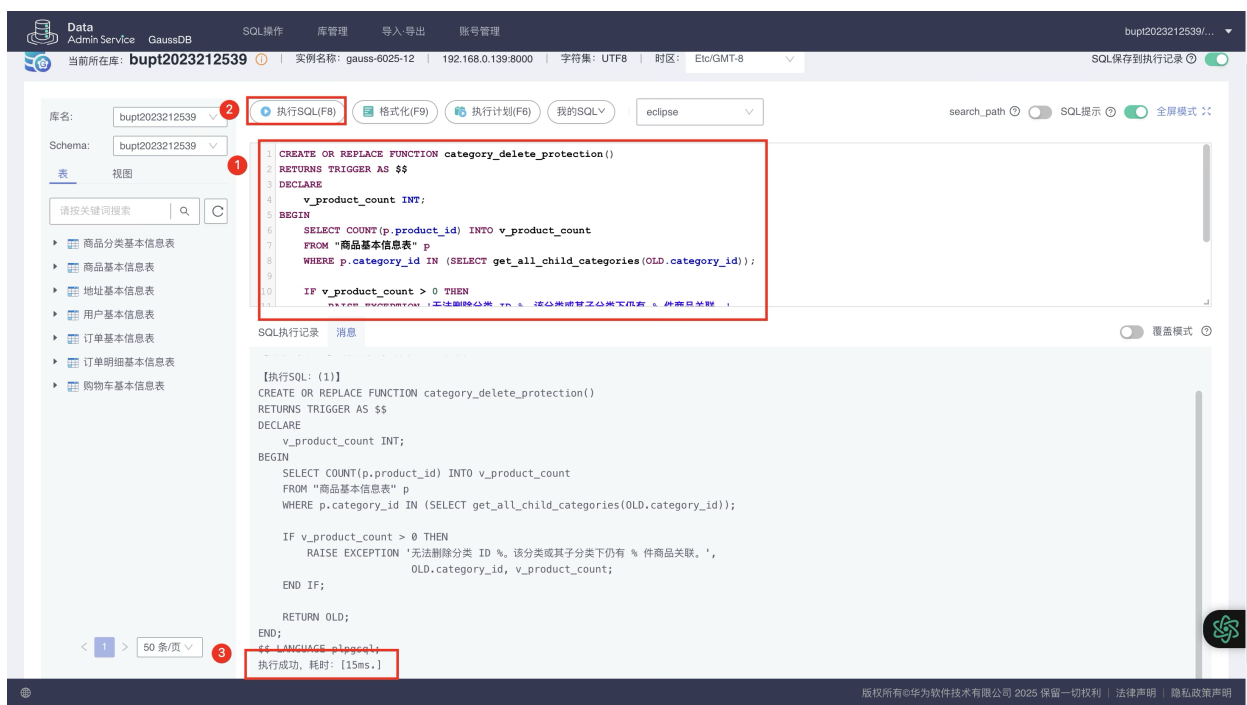


图 4-28 检查受影响分类是否含有商品函数

(2) 创建触发器并绑定函数

创建触发器名为 `category_delete_trigger` 并绑定触发器函数，对应的 SQL 语句见下文，执行结果如图 4-29 所示。

```

1 CREATE TRIGGER category_delete_trigger
2 BEFORE DELETE ON "商品分类基本信息表"
3 FOR EACH ROW
4 EXECUTE PROCEDURE category_delete_protection();

```

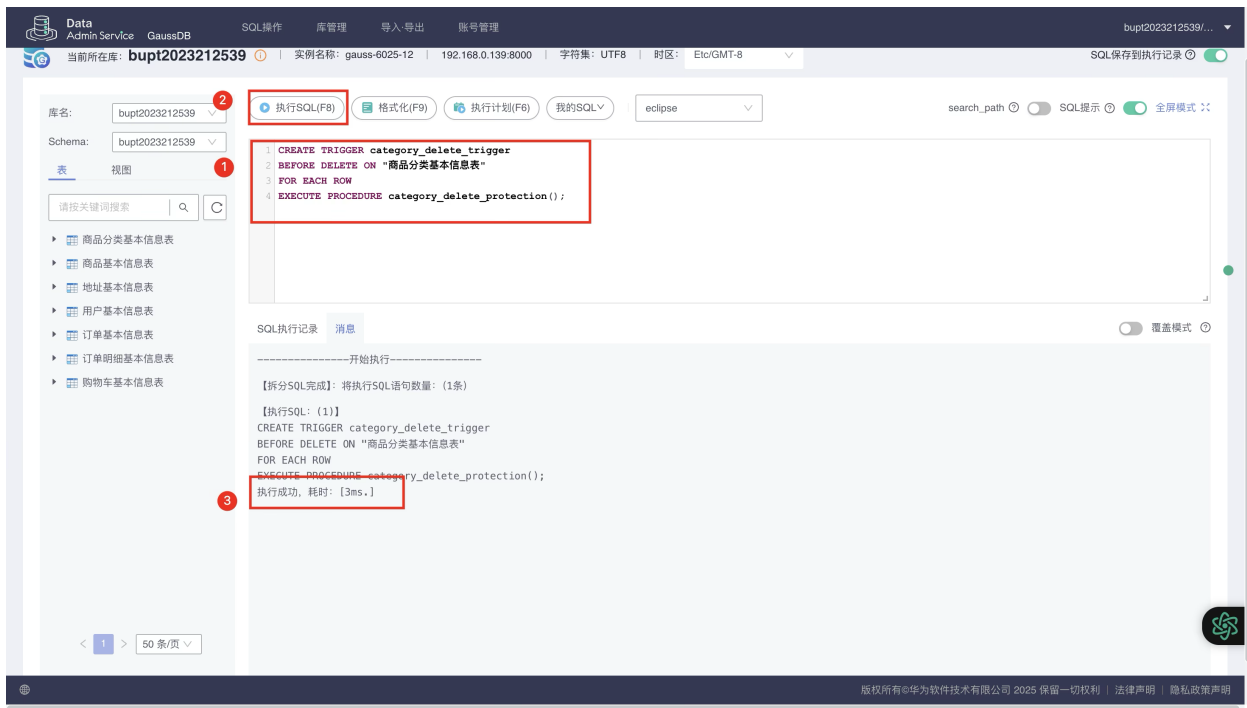


图 4-29 创建触发器

(3) 触发触发器

(3.1) 删除二级分类时，例如，删除二级分类“面食”，对应分类 ID 为 10，其下有商品“红烧牛肉面”等共 2 件商品，对应的 SQL 语句见下文，执行结果如图 4-30 所示，正确触发触发器并调用函数，抛出异常“**ERROR: 无法删除分类 ID 10。该分类或其子分类下仍有 2 件商品关联。**”。

(3.2) 删除一级分类时，例如，删除一级分类“电子产品”，对应分类 ID 为 5，其下包含子分类在内有商品“苹果 iPad Pro”等共 6 件商品，对应的 SQL 语句见下文，执行结果如图 4-30 所示，正确触发触发器并调用函数，抛出异常“**ERROR: 无法删除分类 ID 5。该分类或其子分类下仍有 6 件商品关联。**”。

1	DELETE FROM "商品分类基本信息表" WHERE category_id = 10;
2	
3	DELETE FROM "商品分类基本信息表" WHERE category_id = 5;

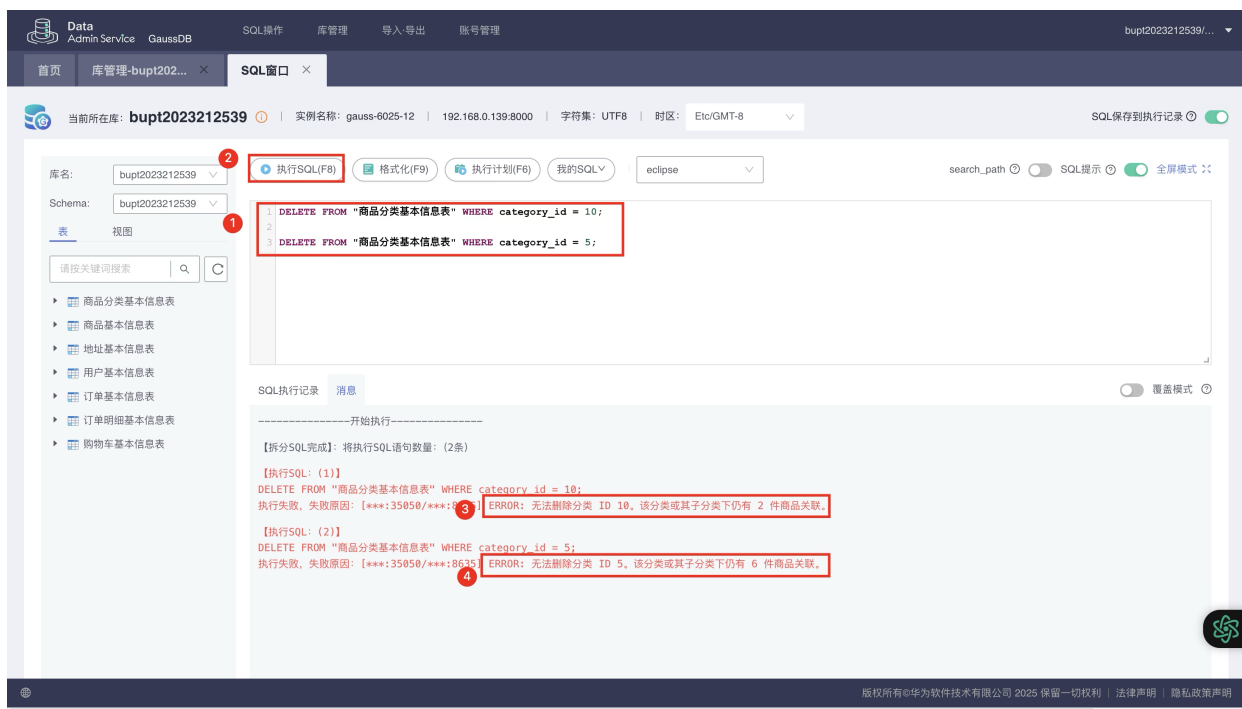


图 4-30 尝试删除其下有商品的一级/二级分类

4.4 管理存储过程

1. 切换到“库管理 → 对象列表”，选择“存储过程”，如图 4-31 所示；
2. 选择 `update_default_address`，单击查看存储过程详情，如图 4-32 所示；
3. 切换到 SQL 窗口，删除存储过程 `add_to_cart`，对应的 SQL 语句见下文，执行结果如图 4-33 所示；

```
1 DROP PROCEDURE add_to_cart;
```

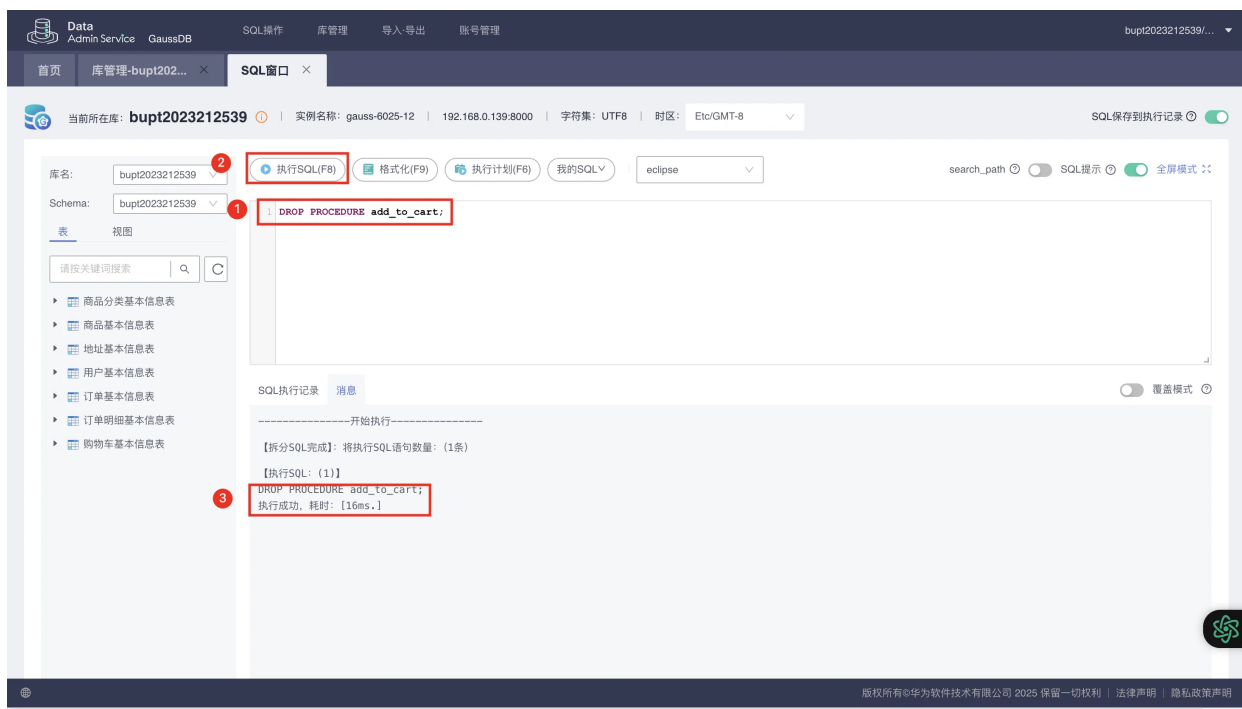



图 4-33 删除存储过程

4.5 管理触发器

1. 切换到“库管理 → 对象列表”，选择“触发器”，选择 `order_item_stock_check_trigger`，单击查看触发器详情，如图 4-34 和图 4-35 所示；
2. 切换到 SQL 窗口，删除存储过程 `product_price_trigger`，对应的 SQL 语句见下文，执行结果如图 4-36 所示；

```
1 DROP TRIGGER product_price_trigger ON "商品基本信息表";
```

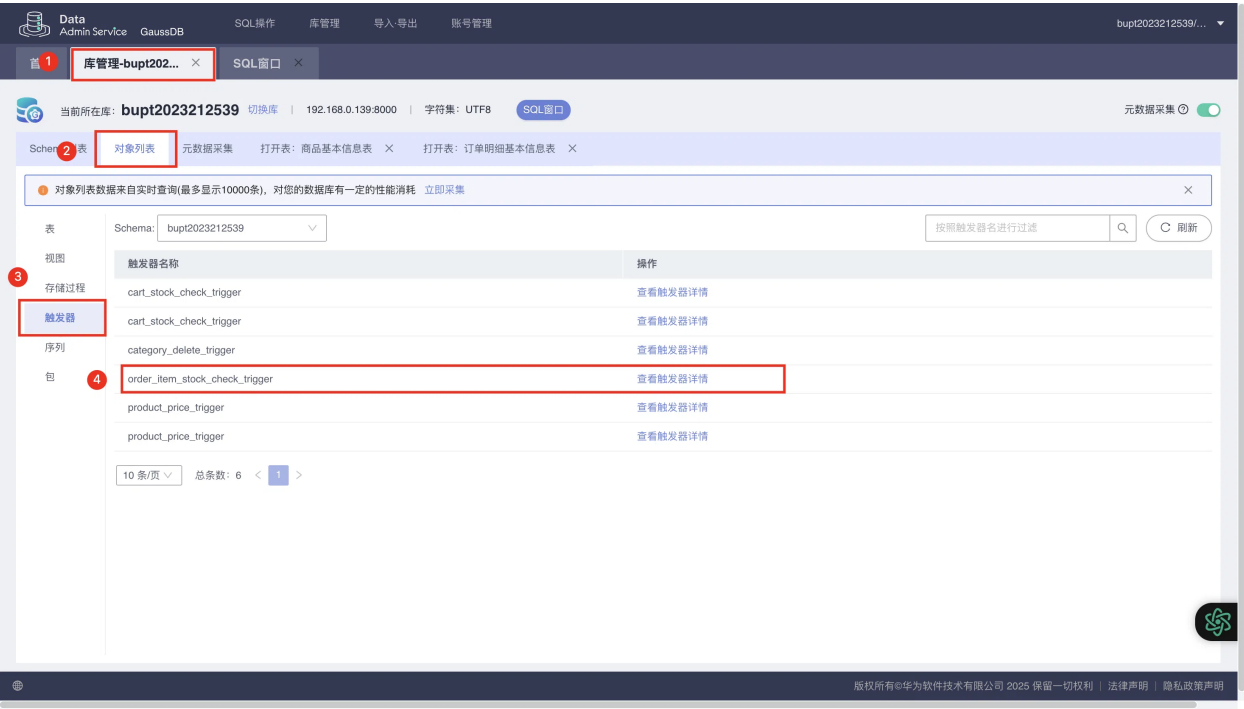


图 4-34 进入触发器管理页面

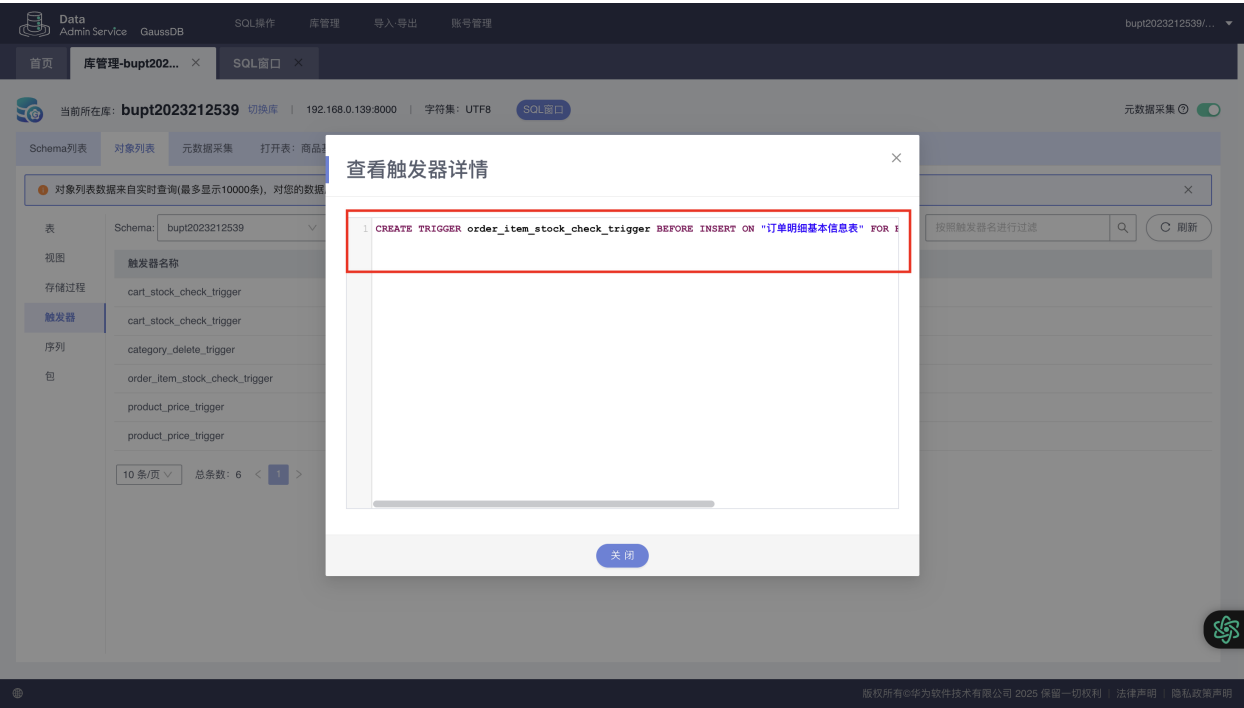


图 4-35 查看触发器详情

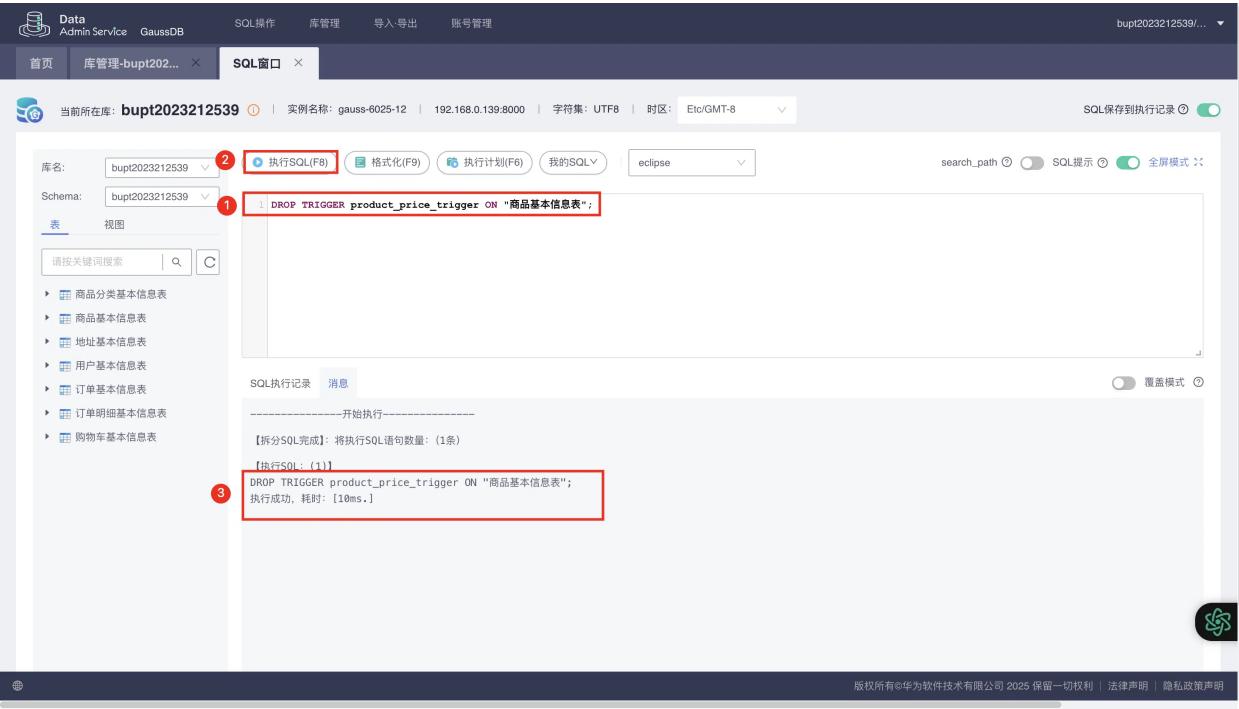


图 4-36 删除触发器

5 实验总结与心得体会

本次实验深入实践了 GaussDB 数据库中的存储过程和触发器的创建与管理，这是实现复杂业务逻辑和数据完整性校验的关键技术。通过完成多个电商场景下的需求，我熟练掌握了 PL/pgSQL 的编程技巧，并对数据库的过程化编程和自动化数据治理有了更深刻的认识。

实验中遇到的问题主要集中于过程化编程的要求和语法，例如在 PL/pgSQL 中，必须使用将系统变量的值捕获到自定义的局部变量中，才能在后续逻辑中使用。以及在 GaussDB 实验环境下，虽然兼容 PostgreSQL，但存储过程/触发器绑定的语法存在一定的差异。

综上，通过本次实验，我深刻感受到了存储过程的强大封装性，掌握了如何使用存储过程来封装复杂的业务逻辑，如递归查找多级分类并批量更新价格、检查-插入的注册逻辑、以及清理超时订单等。存储过程将复杂的 SQL 和控制流集中在数据库层，提高了应用的安全性和效率。同时，通过创建多个 `BEFORE` 触发器，我深入理解了触发器在数据修改前的拦截和校验作用，其在数据完整性上提供了很好的保障作用。

一些对实验的建议：

- 统一并明确序列管理指导：** 序列冲突是实验中反复出现的问题。建议在实验指导中提前放置关于 `SELECT SETVAL(...)` 的标准操作，并解释 `SERIAL` 类型的工作原理。
- 提供更明确的版本语法指导：** 鉴于我们使用的是 GaussDB 版本可能与比较新的 PostgreSQL 的版本存在语法差异，建议在涉及触发器创建时，明确指出语法规则，避免因语法兼容性问题浪费时间。